Astronomy & Astrophysics manuscript no. paper
October 30, 2026
©ESO 2026

Synthesising full-sky maps from scattering covariances on distributed-memory machines

A pyramidal, block-wise explicit gradient in Rust, MPI and CUDA, applied to weak-lensing convergence

P. Campeti[1]⋆, J.-M. Delouis[2], and [TODO: co-authors]

[1] INFN, Sezione di Ferrara, Ferrara, Italy [TODO: verify affiliation]
[2] Laboratoire d'Océanographie Physique et Spatiale (LOPS), Univ. Brest, CNRS, Ifremer, IRD, Brest, France

Received ...; accepted ...

ABSTRACT

Context. Scattering-covariance statistics generate new realisations of a non-Gaussian field from one or a few examples with a small, interpretable set of constraints. On the sphere they are limited by the memory of automatic differentiation: a full-sky gradient at $N_{\rm side} = 2048$ (5×10^7 pixels) fills a large GPU.
Aims. We make the synthesis scale with the nodes of a distributed-memory machine, each process holding a memory bounded by its share of the sky, and we test syntheses of a strongly non-Gaussian field against independent realisations.
Methods. The statistics and their exact gradient are computed without a computational graph, block by block, on a pyramid of anti-aliased HEALPix decimations; the cross-scale terms are carried by decimated moduli stored once per pair of levels, and blocks interact only through fixed halos. The Rust implementation uses MPI (contiguous NESTED ranges, halo exchanges, additive ghost cells, a distributed L-BFGS) and one CUDA device per rank, and matches the reference Python code to 10^{-15}. For weak-lensing convergence the loss adds, in a gaussianised space, the first-order statistics of κ, the Wasserstein distance of the histogram, the power and the kurtosis of every pyramid level, and the scatter of an ensemble.
Results. The evaluation time drops as the inverse of the number of ranks with 75–85% efficiency up to 512 CPU cores at $N_{\rm side} = 1024$, with identical losses for any rank count. Ensembles of ten syntheses of the Takahashi et al. (2017) convergence map at $N_{\rm side} = 1024$ lie within the scatter of ten independent realisations for the angular power spectrum, the per-level variance and kurtosis of κ and the peak counts, but not for the Minkowski functionals of the finer levels. [TODO: 2048 results.]
Conclusions. Scattering-covariance synthesis is no longer bound to the memory of one device, and the same machinery applies to high-resolution, multi-channel and time-dependent geophysical fields such as climate and ocean model outputs.

Key words. Methods: numerical – Methods: statistical – gravitational lensing: weak – large-scale structure of Universe – Techniques: image processing

1. Introduction

Many of the largest numerical data sets of the physical sciences are fields on the sphere. Kilometre-scale global climate and ocean models produce outputs of 10^7 to 10^9 cells per level and per variable, for many variables, every few hours over decades (Stevens et al. 2019; Eyring et al. 2016; Hewitt et al. 2020); reanalyses such as ERA5 hold several decades of hourly global fields (Hersbach et al. 2020); and the weather forecasts produced by machine-learning models are now judged as probabilistic ensembles of global fields (Lam et al. 2023; Bi et al. 2023; Price et al. 2025; Kochkov et al. 2024), some of them on HEALPix grids (Karlbauer et al. 2024). In cosmology, wide surveys measure the weak-lensing convergence and the CMB on the full sky at arcminute resolution, and their non-Gaussian statistics are the target of a growing body of work (Kratochvil et al. 2010, 2012; Petri et al. 2013; Euclid Collaboration: Ajani et al. 2023; Gatti et al. 2024). All these applications need new realisations of the fields: to estimate covariances, to propagate uncertainties, to test pipelines, or to augment a small set of expensive simulations.

Generative models based on scattering transforms (Mallat 2012; Bruna & Mallat 2013) and their covariances (Morel et al. 2025; Cheng et al. 2024) address this need with a different trade-off from deep generative networks. A scattering-covariance model describes a field by a few thousand averaged, interpretable coefficients that capture the coupling between scales and orientations; new realisations are drawn by gradient descent from noise until their coefficients match those of the data, in the spirit of microcanonical models (Bruna & Mallat 2018) and of texture synthesis (Portilla & Simoncelli 2000; Gatys et al. 2015). Such models need no training set: one realisation is enough (Allys et al. 2019, 2020; Cheng et al. 2020; Cheng & Ménard 2021; Jeffrey et al. 2022; Regaldo-Saint Blancard et al. 2021). They have been brought to the sphere for Planck dust polarisation (Delouis et al. 2022), for astrophysical fields at large (McEwen et al. 2022; Mousset et al. 2024), and to emulate CMB systematics from a handful of simulations (Campeti et al. 2025).

The obstacle to larger maps is memory. The synthesis needs the gradient of the statistics with respect to every pixel. With au-

⋆ Corresponding author: paolo.campeti@fe.infn.it

tomatic differentiation the computational graph keeps every intermediate field: on the sphere, the wavelet transforms of every scale and orientation, their moduli, and the second wavelet transforms of the moduli. The Python engine of STL-Dev[1] (STL-Proj 2025) replaced the graph by an explicit, pyramidal and block-wise gradient, and synthesises an $N_{\rm side} = 2048$ map on one 24 GB GPU. Beyond that size, and for several channels or for time sequences, one device is not enough, whatever the implementation.

This paper presents STL-Dev-MPI, a port of that engine to distributed-memory machines: Rust for the core, MPI for the decomposition of the sky between processes, and CUDA for one device per process. The memory of each process is bounded by its share of the sky; the wall-clock time per evaluation drops with the number of processes; and every rank count gives the same result to the last digits. We describe the method (Sect. 2), its parallel implementation on CPUs (Sect. 3) and GPUs (Sect. 4), and the terms we had to add to the loss to synthesise the weak-lensing convergence, a field whose information sits in rare, extended peaks (Sect. 5). We then test ensembles of ten syntheses of the Takahashi et al. (2017) convergence map against the scatter of ten independent realisations of the same simulation suite, with statistics that are not in the loss: the angular power spectrum, the per-level variance and kurtosis, the peak counts, the one-point PDF and the Minkowski functionals (Sect. 6). Section 7 discusses what the distributed implementation opens, in particular for climate and ocean data.

2. Method: a pyramidal, block-wise explicit gradient

A synthesis evaluates the statistics of a map and their gradient with respect to every pixel, hundreds of times. Two properties of the scattering covariance make this possible at any resolution. First, the gradient can be written by hand, family by family, and the intermediate fields it needs can be recomputed when they are needed instead of being stored: the memory of an evaluation is then that of the maps and of a few decimated fields, about 50 bytes per pixel at $L = 4$, instead of the kilobytes per pixel that automatic differentiation keeps (Sects. 2.2 and 2.3). Second, every operation of the statistics is local, except the final pixel means and the decimations that carry the fine moduli to the coarse levels, and both are sums: the statistics and their gradient are sums of contributions of blocks of pixels, each of which needs only the block and a thin halo around it (Sects. 2.4 to 2.6). The sky can then be split between processes like the grid of a finite-difference solver, and each process needs the memory of its share of the sky only.

2.1. Scattering covariance on a HEALPix pyramid

We use the scattering-covariance statistics of STL-Dev, which follow Morel et al. (2025) and Cheng et al. (2024) and were adapted to HEALPix (Górski et al. 2005) by Delouis et al. (2022) and Mousset et al. (2024); we only recall the notation. Let x be a map at resolution $N_{\rm side}$ (level 0) with C channels, and $x_d = D^d x$ its anti-aliased decimations, level d at $N_{\rm side}/2^d$. One decimation step D gives each coarse pixel a Gaussian-weighted, ℓ^1-normalised mean of the fine pixels whose centres lie within $r = 3\,\theta_{\rm fine}$ of its centre ($\theta_{\rm fine}$ the fine pixel size, Gaussian width $r/2$). The wavelet ψ is a complex directional filter with L orientations, applied at every level with the same $K \times K$ stencil in the tangent plane (taps interpolated bilinearly from the four nearest pixels): $z_d = \psi \star x_d$ is a complex field of shape $[C, L, N_d]$, and scale j is computed at level $d(j) = j$. Writing $\langle\cdot\rangle_n$ for a mean over the pixels of level n, the coefficients are

$$S_1(j) = \langle |z_n| \rangle_n, \qquad S_2(j) = \langle |z_n|^2 \rangle_n, \tag{1}$$

$$S_3(j_1, j_3) = \langle [\psi \star m_{d_1,n}]_{\ell_1 \ell_3}\; z^*_{n,\ell_3} \rangle_n, \tag{2}$$

$$S_4(j_1, j_2, j_3) = \langle [\psi \star m_{d_1,n}]_{\ell_1 \ell_3}\; [\psi \star m_{d_2,n}]^*_{\ell_2 \ell_3} \rangle_n, \tag{3}$$

with $n = d(j_3)$ ($n = d(j)$ for S_1, S_2), $d_i = d(j_i)$, cross-channel versions of each, the mean and variance of the map, and the *stored moduli*

$$m_{d,n} = D^{n-d}\,|z_d| \qquad (d < n), \tag{4}$$

the modulus of the wavelet field of level d decimated to level n. Every coefficient is a pixel mean at its *native level* n. With $L = 4$, $K = 5$ and $J = \log_2 N_{\rm side} - 1$ scales, a single-channel map has 11 354 coefficients at $N_{\rm side} = 1024$ and 15 042 at 2048.

The synthesis minimises over the map x

$$\mathcal{L}_S(x) = \frac{1}{N_c} \sum_i \frac{|S_i(x) - S_i(t)|^2}{\sigma_i^2}, \qquad \sigma_i^2 = \frac{\langle |f_i|^2 \rangle - |S_i(t)|^2}{N_{n(i)}}, \tag{5}$$

with t the target, f_i the field whose mean is S_i, and N_n the number of pixels of level n: σ_i^2 is the variance of a mean of N_n independent pixels, estimated on the target (ergodic hypothesis), and $\mathcal{L}_S$ is a reduced χ^2. Without these weights the loss is dominated by the low-order coefficients and S_3, S_4 stay at 1–3σ after 400 iterations; with them every family converges below 1σ.

2.2. What automatic differentiation stores

The earlier syntheses on the sphere (Delouis et al. 2022; Mousset et al. 2024) obtain the gradient from the automatic differentiation of a machine-learning framework, PyTorch (Paszke et al. 2019) or JAX (Bradbury et al. 2018). In reverse mode, every operation of the forward computation records the inputs that its derivative needs, and the record is kept until the backward pass has used it. For the statistics of Sect. 2.1 the record holds at least, for every level n: the wavelet fields z_n and their moduli ($\frac{4}{3}LN$ complex and $\frac{4}{3}LN$ real values over the pyramid, for a map of N pixels and one channel); the decimation chains of the moduli; and the second wavelet fields $\psi \star |z_n|$ and $\psi \star m_{d,n}$ for every finer level $d < n$ and every pair of orientations, $(n+1)L^2$ complex fields of $N_n = N/4^n$ pixels at level n, $\frac{16}{9}L^2N$ complex values in all, which alone is 455 bytes per pixel of the map at $L = 4$. Actual implementations record more (normalised fields, gathered convolution taps, intermediate products), and the convolution geometry (the rotated stencil of every pixel and its interpolation weights) is usually precomputed for the whole sky and kept. In STL-Dev, the automatic-differentiation version of the same statistics holds 371 MB of graph for a map of 49 152 pixels ($N_{\rm side} = 64$, $J = 5$, $L = 2$, $K = 5$), 7.5 kB per pixel, on top of a stored geometry of 3.2 kB per pixel (STL-Proj 2025). Both grow in proportion to the number of pixels: at $N_{\rm side} = 2048$ (5×10^7 pixels) the graph would take about 380 GB at $L = 2$, more at $L = 4$, and the geometry 160 GB. This, and not the arithmetic, is what has limited the syntheses to one device and to moderate resolutions.

2.3. An explicit gradient

Every coefficient is the pixel mean, at its native level, of a product of two fields, $S = \langle a\, b^* \rangle_n$, or of a modulus. Its derivative is

[1] https://github.com/STL-Proj/STL-Dev

known in closed form: a cotangent on S gives, pixel by pixel, a cotangent on a proportional to b and a cotangent on b proportional to a, both divided by N_n. The chain then runs backwards through operations whose adjoints are explicit and sparse: the second wavelet convolution (its adjoint scatters every interpolated tap back to the four pixels it came from), the decimations ($D^\top$), the modulus ($\bar{z} = \overline{|z|}\, z/|z|$), the first wavelet convolution, and finally the decimations that made x_n from x. Each adjoint costs about as much as the operation itself; the gradient is checked against finite differences, and the explicit gradient of the reference against its automatic differentiation (Table 1).

The fields that the backward pass needs are not kept: they are recomputed from the maps when the backward pass reaches them, block by block (Sect. 2.5). This is the checkpointing strategy of automatic differentiation pushed to its limit, and it costs one extra evaluation of the forward fields. Only two kinds of quantities must survive from the forward to the backward pass, because they cannot be recomputed locally: the maps of all levels, $x_d = D^d x$ ($\frac{4}{3}N$ values per channel), and the stored moduli $m_{d,n}$ of Eq. (4). The stored moduli are the only carriers of information across scales: S_3 and S_4 at level n read the moduli of all the finer levels, but only after their decimation to level n, which divides their size by 4^{n-d}. Summed over the pairs $d < n$, they hold $\frac{4}{9}LN$ values per channel, less than the moduli of the finest level alone. With the cotangents of both, the state of an evaluation is $2(\frac{4}{3} + \frac{4}{9}L)NC$ real values, 50 bytes per pixel and map at $L = 4$, against the 7.5 kB per pixel of graph and 3.2 kB per pixel of geometry above. The convolution geometry is recomputed for each block and never stored (Sect. 2.7).

2.4. Locality and blocks

Every operation of the chain is local. A wavelet convolution reads the $K \times K$ stencil of a pixel, each tap interpolated from its four nearest pixels; a decimation step reads the fine pixels within $r = 3\,\theta_{\rm fine}$ of a coarse centre; moduli and products act pixel by pixel. A field on a set F of pixels of level n therefore needs its inputs only on F plus a *halo* of a few pixels of that level, whatever the size of the map. Only two operations are not local, and both are sums. The coefficients are pixel means: they are sums of partial sums over any partition of the native level into blocks. The stored moduli carry $|z_d|$ across $n - d$ levels; since D is linear,

$$m_{d,n} = \sum_k D^{n-d}\left(|z_d|\,\mathbf{1}_{F_k}\right), \tag{6}$$

where $\{F_k\}$ is a partition of level d into blocks of b pixels. Each *piece* needs z_d on F_k only, which needs x_d on F_k plus a convolution halo, and its decimation chain is supported on a cone that grows by a decimation halo per level (Fig. 1). The gradient goes back through the transposed pieces, the derivative of the modulus and the adjoint of the wavelet convolution, on the same supports.

Halos are counted in pixels of their level: $(K-1)/2 + 2$ for a convolution, $\lceil r/\theta \rceil + 1$ for a decimation, doubled for the pieces and their transposes so that every coarse row touched by a block sees its whole cone. Fine-level dilations are computed at the parent level (k coarse rings cover at least $2k$ fine pixels). The blocks are NESTED ranges, compact on the sphere, so a halo is a thin ring around a patch.

The statistics are thus a sum over blocks of partial sums, each computed from the block and its halo, and the gradient, which runs the same chain backwards, is a sum over blocks of contributions, each written into the block and its halo. Between two blocks nothing is kept but the maps, the stored moduli and their cotangents: the fields of a block, z_n, its modulus and the second wavelets on F plus a halo, have a size set by b, not by N. The result does not depend on the blocks: any b, from 64 pixels to whole levels, gives the full-sky statistics and gradient to rounding (Sect. 2.8).

2.5. Sweeps

One evaluation of the loss and of its gradient is two sweeps over the levels.

Upward sweep, $n = 0, \ldots, J-1$: for every block F of level n, (i) the wavelet field z_n on F plus a halo, from x_n on F plus two halos; (ii) the second wavelets of $|z_n|$ (same level) and of $m_{d,n}$ ($d < n$, read from the stored maps) on F; (iii) the partial sums of every coefficient of native level n; (iv) the piece $|z_n|\mathbf{1}_F$ pushed down to every coarser level $n' > n$ and added to $m_{n,n'}$.

Downward sweep, $n = J-1, \ldots, 0$: for every block of level n, the fields of the upward sweep are recomputed; the coefficient cotangents $2(S_i - S_i(t))/(N_c\sigma_i^2)$ (real and imaginary parts for complex coefficients) are applied; the cotangents $g_{d,n}$ of the stored moduli of the finer levels are accumulated (they are complete once the coarser levels are done); the block's own chain is reversed from $g_{n,n'}$, $n' > n$, through the transposed pieces; then the modulus derivative and the wavelet adjoint give the cotangent of x_n on the block.

Finally the map cotangents of all levels are brought back to level 0 by Horner's scheme through the transposed decimations, $\nabla_x \mathcal{L} = \sum_d (D^\top)^d\, g_{x_d}$. Everything is in double precision and every summation order is fixed.

2.6. Why the gradient parallelises

Locality turns the gradient into a domain-decomposition problem, like the solution of a partial differential equation on a grid. Let the pixels of level 0 be split between P processes as compact patches, each process owning the parents of its pixels at every level. A process can compute the contributions of all its blocks, in both sweeps, if it holds at every level its own pixels plus a ring of halo pixels, and if it knows the cotangents of the N_c coefficients, which are global. Three kinds of communication are enough (Sect. 3): halo exchanges, which copy into the ring the values owned by the neighbouring processes (maps, stored moduli, cotangents); ghost reductions, which send the contributions that a process wrote into its ring (the edges of its pieces and of its transposed convolutions) to their owners, where they are added; and one global sum of the N_c partial sums per evaluation, after which every process knows the statistics, the loss and the cotangents of the coefficients. Nothing else couples distant pixels.

Three consequences follow. (i) The memory of a process is its share of the state of Sect. 2.3, $(50 + 192)\,N/P$ bytes at $L = 4$ with the L-BFGS history, plus its halo ring and the transients of one block. The ring scales as the perimeter of the patch: at $N_{\rm side} = 1024$ on 64 processes a process owns 196 608 pixels of level 0 and holds 13% more with its halos (Sect. 3.5). (ii) The communication of a process also scales as its perimeter, $\propto (N/P)^{1/2}$ at fixed level, plus a vector of N_c coefficients, independent of N. (iii) There is no limit on the size of the map other than the number of processes: multiplying N by four (one more level of $N_{\rm side}$) with four times as many processes leaves the memory and the work of each process unchanged. A synthesis that does not fit one device runs on several, with the same result to rounding: $N_{\rm side} = 2048$ at $L = 4$ does not fit one 16 GB GPU (its

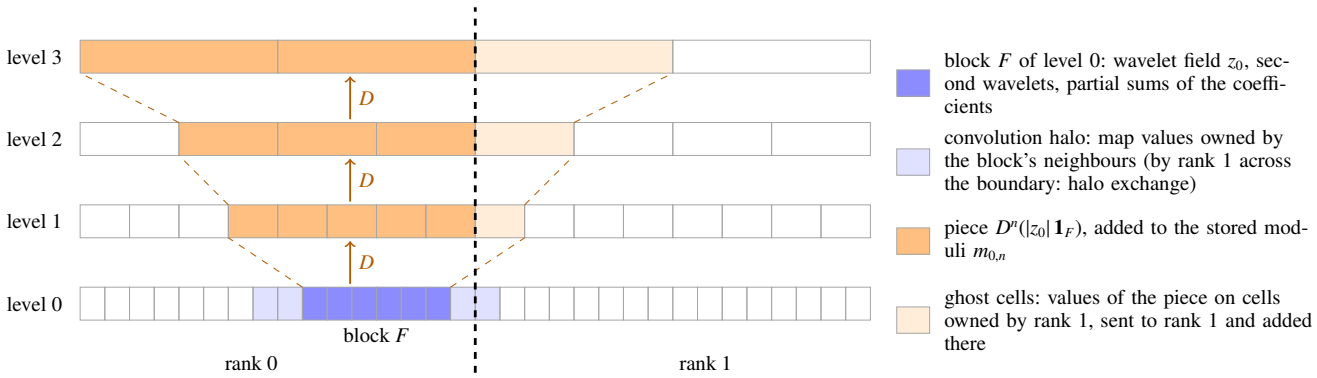


Fig. 1. Locality of the statistics, in one dimension (a cartoon of the sphere, where each level has four times fewer pixels than the level below). The wavelet field of a block F of level 0 needs the map on F and a convolution halo. Its modulus restricted to F is decimated level by level (the piece of Eq. 6), on a support that grows by a decimation halo per level (dashed), and is added to the stored moduli. With the sky split between processes, a halo cell owned by another process is filled by a halo exchange, and a piece's values on cells owned by another process are sent to it and added (ghost reduction). The gradient runs the same supports backwards: transposed pieces, modulus derivative, convolution adjoint.

level fields and decimation rows alone take about 24 GB, and still 15 GB with the wavelet fields recomputed per block, whatever the form of the convolution) but runs on two, and $N_{\text{side}} = 4096$ runs on 16 GPUs of 16 GB (Sect. 4). The coarsest levels hold fewer pixels than there are processes, so their halos may cover them entirely; they are small ($N/4^d$ pixels), and holding them in full costs little. Since the global sums are made in a fixed rank order, the result does not depend on P.

2.7. Memory

Per block the engine holds fields of size $|F$ + halo$|$ times CL, and the convolution geometry of the block's rows: the rotated stencil of each output row, interpolated on the four nearest pixels, $4GK^2$ entries per row for G gauges, computed on the fly and never stored globally (a stored geometry costs kilobytes per pixel). Globally it holds the maps of all levels and their cotangents ($2 \times \frac{4}{3}NC$ values), the stored moduli and their cotangents ($2 \times \frac{4}{9}NCL$, real), and the history of L-BFGS ($2(h+2)N$ values for a history h, work vectors included). For B maps synthesised together, every term is multiplied by B. At $L = 4$ and $C = 1$ the maps, the stored moduli and their cotangents take $8(8/3 + 8L/9) \simeq 50$ bytes per pixel and map before the halos (0.6 GB at $N_{\text{side}} = 1024$, 2.5 GB at 2048), and L-BFGS with $h = 10$ adds 192 bytes per pixel and map. The GPU engine (Sect. 4) also keeps the level fields z_n and $|z_n|$ whole, about 220 bytes per pixel and map in all (2.8 GB at $N_{\text{side}} = 1024$, 11.1 GB at 2048); recomputing them per block, as the CPU engine does, brings this to 60 bytes (preliminary). The point of the decomposition (Sect. 3) is that all these terms are divided by the number of processes.

2.8. Validation against the reference

STL-Dev-MPI was written against fixtures dumped from STL-Dev; every test compares the two (Table 1). The geometry agrees to 10^{-12} or to the rounding of the reference itself (its `acos` differs from the C library's by one ulp on about 10% of the inputs, amplified by $1/\sin\theta$ near the poles). The statistics agree to 7×10^{-16} and the gradients to 9×10^{-16}, over 8 cases with $N_{\text{side}} = 16$ and 32, $J = 3$ and 4, $L = 2$ and 4, kernels 3 and 5, 1 and 2 channels, both definitions of S_3/S_4, and blocks from 64 pixels to whole levels. The L-BFGS optimiser is a line-by-line port of `torch.optim.LBFGS` (Liu & Nocedal 1989; Paszke et al. 2019) with its strong Wolfe line search (Wolfe 1969; Nocedal & Wright 2006): a 20-iteration synthesis of the WebSky convergence map at $N_{\text{side}} = 64$ reproduces the 25 loss values of STL-Dev at the printed precision, with any number of ranks.

Table 1. Agreement of STL-Dev-MPI with the STL-Dev reference.

Quantity	Max. difference
Pixel indices, neighbours, decimation sparsity	exact
Pixel centres, stencil rotations	10^{-14}
Ring interpolation, decimation weights	10^{-12}
Block wavelet, adjoint, pieces	8×10^{-14}
Statistics (values $O(1)$)	7×10^{-16}
Map and band gradients	9×10^{-16}
Gradient vs finite differences (relative)	2.5×10^{-9}
2, 4 ranks vs 1 (threads, `mpirun`)	7×10^{-16}
GPU vs CPU statistics	6×10^{-16}
L-BFGS losses vs STL-Dev[a]	2.3×10^{-10}

Notes. [a] $N_{\text{side}} = 64$, 25 evaluations.

3. Distributed memory

3.1. Decomposition and halos

The sky at level 0 is split into contiguous NESTED ranges, one per rank, aligned on 4^k pixels with k as large as the rank count allows. A rank owns the cell c of level d when its first descendant at level 0, $c\,4^d$, is in its range. When the ranges are aligned on 4^{J-1}, every rank owns the parents of all its cells and its pyramid is aligned: x_d, $m_{d,n}$ and their cotangents are sliced the same way at every level. With more ranks than cells at the coarsest levels, some ranks own no coarse cell; nothing else changes.

Each rank holds, at every level, its owned cells plus the halo cells that its blocks touch: the convolution domains of its blocks, the decimation domains of the blocks of the next level, the parents of its cells, and the supports of the decimation chains of its finer blocks (Sect. 2.4). These layouts are computed once. The neighbours are found with HEALPix neighbourhoods at the parent level, and each rank tells the owners of its halo cells which cells it needs; the counts on both sides are checked once at setup.

Two kinds of exchange cross rank boundaries during the sweeps (the ghost-cell pattern of domain decomposition;

Smith et al. 1996): *halo exchanges*, where owners send the current values of the cells their neighbours hold (maps after each decimation, stored moduli before they are read, cotangents before the transposed decimations), and *ghost reductions*, where the contributions that a rank's pieces and transposed pieces wrote into cells owned by a neighbour are sent to the owner and added. Owners add the contributions of the other ranks in rank order, so the result does not depend on the arrival order of the messages. Both are point-to-point messages with neighbours only, one tag per exchange; the exchanges of all the stored moduli of a level are batched, so an evaluation needs about $5L$ exchanges with each neighbour (26 at $N_{\rm side} = 64$, about 56 at 2048, instead of 58 and 218 before batching). The partial sums of the coefficients are gathered and broadcast in rank order (one vector of N_c complex numbers per evaluation), so that the statistics, the loss and every decision of the optimiser are identical on every rank and do not depend on the number of ranks: 1, 2 and 4 ranks agree to 7×10^{-16}, and the $N_{\rm side} = 1024$ runs of Table 2 on 24 to 192 ranks and 1 to 4 nodes give the same losses to 16 digits.

No rank ever holds a full-sky map: the target and the initial map are read by owned slices, the white noise of the initial map is drawn per pixel from (seed, pixel) so that it does not depend on the decomposition, and the outputs are written by slices at their offsets.

3.2. Distributed L-BFGS

The L-BFGS history vectors are distributed like the map. The two-loop recursion and the strong Wolfe line search need only dot products and norms, which are global sums of per-rank partial sums (in rank order). The port of `torch.optim.LBFGS` goes through an abstract reduction, so the same code runs on one process, on thread ranks (for the tests), and under MPI.

3.3. Ensembles

An ensemble of B syntheses is optimised jointly with the loss on the mean of their statistics,

$$\mathcal{L}_S = \frac{1}{N_c} \sum_i \frac{|\bar{S}_i - S_i(t)|^2}{\sigma_i^2 / B}, \qquad \bar{S}_i = \frac{1}{B} \sum_{m=1}^{B} S_i(x_m), \tag{7}$$

so that the ergodic scatter of the target goes into the differences between the members instead of being forced to zero in each of them. The mean alone leaves the scatter free (at $N_{\rm side} = 64$ every member of a 4-member ensemble ends 3–4.6σ from the target, where an ergodic realisation sits about 1σ away); a second-moment term holds it at the ergodic variance:

$$\mathcal{L}_D = \frac{w_D}{N_c} \sum_i \left(\frac{\mathcal{D}_i}{\sigma_i^2} - 1 \right)^2, \qquad \mathcal{D}_i = \frac{1}{B-1} \sum_m |S_i(x_m) - \bar{S}_i|^2. \tag{8}$$

With $w_D = 1$ the members end at about 1σ from the target, with a scatter at the ergodic variance (0.94–1.12 per family at $N_{\rm side} = 64$).

The ranks are split into G groups of consecutive ranks (`MPI_Comm_split`); a group shares the sky of B/G members, which its engine evaluates as a batch, and the groups exchange one vector of N_c complex partial sums (and one of N_c reals for Eq. 8) per evaluation. $G = 1$ costs no extra communication and suits small maps; $G = B$ gives each member the memory and time of a single synthesis on P/B ranks or devices. Any G gives the same synthesis up to the order of the sums (10^{-12} relative after 30 iterations at $N_{\rm side} = 64$).

3.4. Overlap mode

An alternative decomposition (option `--overlap`, CPU engine) trades computation for communication. Every rank evaluates, for the fine levels ($d < n_c$), its owned blocks and the halo blocks they depend on, on extended regions, so that the sweeps need no exchange; the coarse levels ($d \geq n_c$), small, are held whole and evaluated by rank 0. An evaluation then needs only the level-0 map halo, one global sum of the coarse stored moduli and of the map of level n_c, the coefficient sum, one broadcast of the coarse cotangents and the level-0 ghost reduction of the map cotangent. It agrees with the standard mode to 7×10^{-16} on every fixture with 1, 2 and 4 ranks. [TODO: timings of the overlap mode at scale.]

3.5. Performance

Table 2 and Fig. 2 give the time of one evaluation (statistics and gradient) on three clusters: Datarmor (Ifremer; 2× Xeon E5-2680 v4 per node, Intel MPI), the hpc cluster (2× AMD EPYC 9655 per node, OpenMPI 5) and Irène Rome (TGCC; AMD EPYC 7H12, OpenMPI 4.1). At $N_{\rm side} = 64$ the time drops by 10 at 28 ranks on Datarmor and by 11.5 at 128 ranks on hpc, where it saturates (a rank owns 384 pixels of level 0 at 128 ranks). At $N_{\rm side} = 1024$ the efficiency is 75% from 24 to 192 ranks on one hpc node, and spreading the 192 ranks over 4 nodes costs 5%; on Irène it is 85% from 128 to 512 ranks. $L = 4$ costs 1.3 to 1.5 times $L = 2$ (12.8 s per evaluation on 512 Irène ranks at $N_{\rm side} = 1024$, 33.1 s on 160 ranks). Ensembles add the members' work and one exchange of partial sums between groups: a production ensemble of 10 maps at $N_{\rm side} = 1024$, $L = 4$, with all the terms of Sect. 5, runs at 128.4 s per evaluation (all members) on 640 Irène cores in 10 groups of 64 (316 evaluations in 11.3 h, the loss going from 5.7×10^4 to 2.46), and at 258 s on 400 Datarmor ranks in 10 groups of 40 (300 more iterations, 311 evaluations in 22.4 h, the loss going on to 0.80); at $N_{\rm side} = 2048$, four times the pixels, the same ensemble takes about 1070 s per evaluation on the same Datarmor cores; the same ensemble without the κ terms ran at 149 s on 160 hpc cores in 10 groups of 16. The state per rank (maps, stored moduli, cotangents, L-BFGS history) shrinks as $1/P$: 123 MB at 24 ranks, 17 MB at 192 for $N_{\rm side} = 1024$, $L = 2$. In the Irène ensemble a rank owns 196 608 cells of level 0 and holds 221 760 with its halos; its state is 52.5 MB, of which 37.7 MB of L-BFGS history, 9.7 MB of stored moduli and cotangents and 5.1 MB of maps and cotangents (the halos weigh most on the stored moduli, which live on the coarse levels). What remains is the transients of one block (80–160 MB for blocks of 16 384 pixels, halved with blocks of 4096) and the MPI runtime (20–30 MB per rank for Intel MPI, 200–350 MB for OpenMPI 5). For comparison (on another machine), STL-Dev on the $N_{\rm side} = 64$ case takes 12.6 to 18 s per evaluation on 4 CPU threads with a peak of 1.9 GB, against 8.1 s and 103 MB for one STL-Dev-MPI rank on Datarmor.

4. GPU engine

The GPU engine runs the same sweeps on one CUDA device per MPI rank, with the CPU decomposition, exchanges, objective and L-BFGS. Its kernels are compiled at start-up by NVRTC to a native binary for the device, with fused multiply-adds dis-

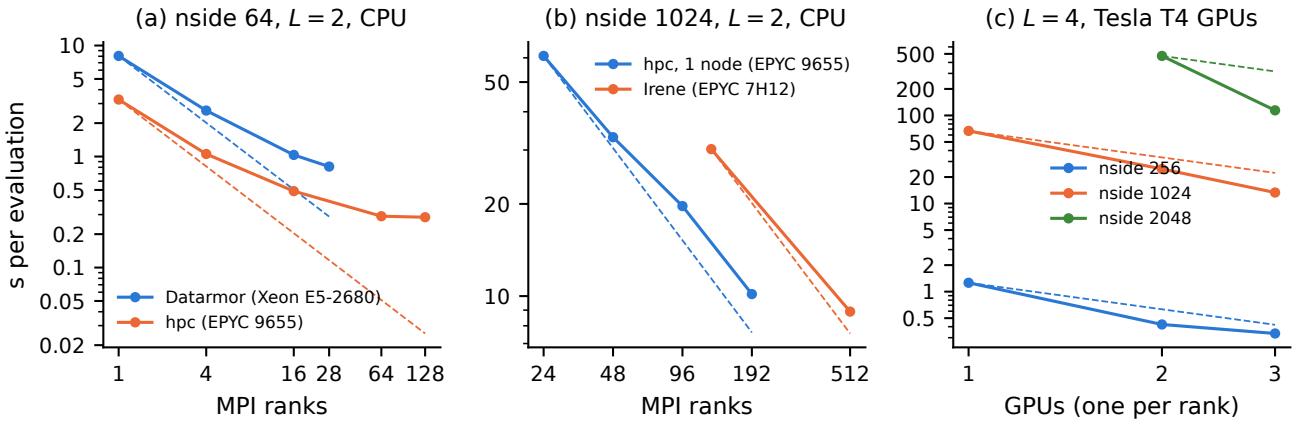


Fig. 2. Strong scaling of one evaluation (statistics and gradient). (a) $N_{\rm side} = 64$, $L = 2$, on Datarmor and hpc; (b) $N_{\rm side} = 1024$, $L = 2$, on one hpc node and on Irène Rome; (c) $L = 4$ on one to three Tesla T4 GPUs (garoupe) at $N_{\rm side} = 256$, 1024 and 2048 (one T4 cannot hold $N_{\rm side} = 2048$), with the bound stencils; the diagonal form of the convolution, now the default, makes one T4 2.5 and 2.7 times faster at $N_{\rm side} = 256$ and 1024 (Sect. 4). Dashed lines: ideal scaling from the first point.

Table 2. Seconds per evaluation on CPUs (kernel 5, blocks of 16 384 pixels, one map).

$N_{\rm side}$	L	Ranks	Machine	s / eval.
64	2	1	Datarmor	8.06
64	2	28	Datarmor	0.81
64	2	1	hpc	3.26
64	2	128	hpc	0.28
1024	2	24	hpc, 1 node	60.9
1024	2	192	hpc, 1 node	10.2
1024	2	4×48	hpc, 4 nodes	10.6
1024	2	128	Irène Rome	30.2
1024	2	512	Irène Rome	8.9
1024	4	512	Irène Rome	12.8
1024	4	160	[TODO: machine]	33.1

abled and IEEE division and square root, and loaded through the `cudarc` crate (Lowman 2026), so the code builds without a CUDA toolkit. Every expression is evaluated in the order of its CPU counterpart, there are no floating-point atomics, and the results are identical from run to run; they differ from the CPU engine only through the transcendental functions and a few documented summation orders (statistics within 6×10^{-16}, level-0 gradient within 9×10^{-13} of its maximum). With one rank, nothing leaves the device; with several, the halo cells are exchanged through the host.

Blocks and geometry. On the device a level is processed by blocks of up to 2^{20} owned cells, sized from the free memory, hundreds of times larger than on the CPU; the layouts do not depend on the block size, so device blocks read the same halos. The convolution geometry (rotated stencils, ring bilinear interpolation, binding to the rank's grid) is computed by a kernel per output row and gauge, then transposed by grid cell so that the adjoint convolution is a gather in a fixed order. Within a memory budget, the geometry of the blocks is kept between sweeps either whole (24 bytes per entry) or as its transposition only (4 bytes per entry, the weights being recomputed). All the GPU timings and memory peaks of this paper (this section, Fig. 2, Tables 3, 5 and 6) were measured with these bound stencils, except those marked DIA. The engine now uses by default the diagonal form of the convolution described at the end of this section, and keeps the bound stencils as an option; below, each GPU figure is followed by what the diagonal form changes, measured or expected. It changes the convolution geometry of the GPU engine only: the CPU engine, its timings (Sect. 3.5) and the CPU ensembles of Table 6 are unaffected.

Decimations. Instead of pieces, the device applies the full-sky decimation rows of the rank's owned coarse cells to the halo-exchanged finer field; the reverse chain is accumulated from the coarsest level by Horner's scheme. These are the values of the pieces, summed in another order.

One engine per rank. A synthesis evaluates several objectives on the same decomposition: the target statistics, the synthesis itself and, for weak lensing (Sect. 5), the statistics of $\kappa = f(g)$. One engine per rank is shared by all of them (topology, decimation rows, geometry cache and level fields), since an evaluation of one objective ends before the next starts. A plan without S_3 or S_4 neither computes nor allocates the moduli, the stored moduli and their chains. The level fields of the full plan then take $8BCH_0(8/3 + 56L/9)$ bytes for H_0 cells held at level 0 and B members, and the geometry cache takes most of what remains. Before this change, each objective had its own engine, sized from the memory free when it was built: a 10-member ensemble at $N_{\rm side} = 1024$ with the κ term ran out of device memory during setup, on an H200 (141 GB) as on an A100 (80 GB), unless the geometry cache of every engine was capped. On the A100, capped at 6.3 GB per engine, an evaluation took 86.6 s after a setup of 156 s, with a 70.2 GB peak (3 evaluations, Table 3). With one engine the same ensemble runs uncapped on the same device in 64.4 s per evaluation (26% less) after a setup of 86 s (45% less: one topology instead of one per engine), with the same losses to 3×10^{-16}; its peak, 62.8 GB, is lower although its single cache (28.4 GB) is larger than the caps of all the engines together. The peak is the level fields (27.7 GB by the formula above), the decimation rows and topology (3.4 GB), the transients of a block (2.4 GB) and the cache, which takes the rest. The device is busy only 18% of the time; the rest is host work, with the 6 threads of that node (the objective's terms, since moved to the device, below; not re-measured on the A100). With the diagonal form of the convolution (below), the 28.4 GB of transposed stencils are no longer needed: the masks of all rows take about 0.3 GB and the rest of the budget holds whole coefficients, so most rows no longer have their weights recomputed at every sweep, while the peak, set by the budget, stays (expected from the per-row sizes; not re-measured). At $N_{\rm side} = 2048$, five members now fit

on the A100, where they took 139 GB of an H200 before: their fields (55.5 GB) and decimation rows (13 GB) leave 8.6 GB to the cache, a small part of the 108 GB of the transposed geometry, so the geometry of every block is rebuilt at every sweep, 924 s of device time over the three evaluations against 73 s for the convolutions, adjoints, decimations and coefficients together; an evaluation takes 395 s. This is where the diagonal form gains most: the masks of all rows (about 1 GB) fit in the cache, and each sweep only recomputes coefficients instead of rebuilding and sorting the stencils; the same ensemble then takes 163 s per evaluation, 2.4 times less, with losses equal to 2.3×10^{-15} (preliminary, measured with the reverse chain applied per block, below, which lowers its fields to 47.3 GB and its peak to 75.1 GB). Ten members at $N_{\rm side} = 2048$ do not fit on 80 GB with these fields held whole: they take 111 GB, 77% of which are z_n and $|z_n|$ with the reverse chain. The diagonal form does not change the level fields, hence neither this limit nor the share of the fields in the peaks above; recomputing z_n and $|z_n|$ per block does, and ten members then fit (below). On a 6 GB laptop GPU, a 2-member ensemble at $N_{\rm side} = 128$ with the κ term, which needed such a cap, now runs 2.6 times faster per evaluation, with identical losses.

Performance (Table 3; bound stencils unless stated). On the Tesla T4, whose double-precision throughput is 1/32 of its single precision, three devices take 13.3 s per evaluation at $N_{\rm side} = 1024$, $L = 4$, and 114 s at 2048, where the geometry no longer fits and is rebuilt at each sweep; three T4s are twice as fast as a 192-core EPYC node at $N_{\rm side} = 1024$, $L = 2$. The scaling from 1 to 3 T4s is superlinear while the geometry cache covers more of each rank's rows. With the diagonal form, one T4 is 2.5 times faster at $N_{\rm side} = 256$ and 2.7 times at 1024; the runs on 2 and 3 T4s, and at $N_{\rm side} = 2048$, where the rebuilds dominate and the gain on an A100 is largest, have not been repeated, and the comparison with the EPYC node, which the diagonal form does not touch, can only move further in favour of the GPUs. On H200 GPUs, an ensemble of 10 syntheses at $N_{\rm side} = 1024$ runs on a single device, and a single map at $N_{\rm side} = 4096$ (2×10^8 pixels, $J = 11$) takes about 325 s per evaluation on one device and 67.9 s on two, a superlinear gain as on the T4s, where it comes from the geometry cache covering more of each rank's rows. In a three-evaluation test on two H200s (60 s per evaluation), the geometry timer of the first device reached 185 s while the three evaluations took 181 s; with the diagonal form these rebuilds should mostly go, on one device as on two (expected from the A100 runs; not re-measured). The same $N_{\rm side} = 4096$ synthesis runs on 16 V100s with 16 GB each (25.2 s per evaluation, 13.9 GB per device): small devices are usable as long as there are enough of them. On Irène, a node of 4 V100s is worth about 1024 AMD Rome cores (23.4 against 28.8 s per evaluation for the same synthesis at $N_{\rm side} = 2048$), a comparison made before the diagonal form, which speeds up the GPUs only. [TODO: repeat with the diagonal form: the T4 scaling, the ensembles of Table 3 at $N_{\rm side} = 512$ and 1024, and the $N_{\rm side} = 4096$ runs; final numbers for the recomputed fields once merged.]

A diagonal form of the convolution. In the coordinates of a HEALPix face, extended across its edges and corners with the neighbour tables of the HEALPix library, every cell that a row reads through its rotated and interpolated stencil (kernel 5) lies at one of 81 offsets within ±5 pixels of the row, the same at every $N_{\rm side}$ from 16 to 1024. The convolution is then exactly a diagonal (DIA) operator, $y_k = \sum_d a_{d,k}\, x_{k+o_d}$, whose coefficients fold together the kernel, the rotation and the bilinear weights: no position is stored, and the adjoint gathers from the opposite offsets, with neither transposition nor sort. A row uses about 44 of these diagonals over its 4 gauges; keeping a mask of them and their coefficients only takes 3152 bytes per row at $L = 4$, instead of 9608 for the bound stencils and their transposition, and 16 bytes between sweeps instead of 1608. On a block of 262 144 rows on a T4 ($L = 4$), the forward convolution is 1.6–1.75 times faster, the adjoint 1.7–2.0 times and the first build of the geometry 1.9 times. The results differ from those of the stencils by summation orders only: on the test fixtures, the statistics by 5×10^{-17} and the gradients by 5×10^{-15} (relative). It is now the default form of the GPU engine. On one A100 (one map, 4 evaluations from white noise, one run per case; Table 3), an evaluation takes 3.04 instead of 4.87 s at $N_{\rm side} = 1024$ (1.6 times faster) and 17.9 instead of 46.3 s at $N_{\rm side} = 2048$ (2.6 times), with losses equal to 2×10^{-15} relative. At $N_{\rm side} = 2048$ the transpositions of the stencils do not all fit in the cache, so most blocks are rebuilt and sorted at every sweep, and the geometry takes 168 of the 185 s of the run; the diagonal form keeps the masks of all blocks and recomputes only their coefficients, and the geometry takes 42 of 72 s. The memory peak does not drop, since the cache budget fills the device in both forms: the diagonal form holds more geometry in it. On one T4 the gain is 2.5 times at $N_{\rm side} = 256$, where the whole geometry now fits and the peak falls from 9.8 to 4.2 GB, and 2.7 times at 1024. The diagonal form leaves unchanged the level fields, $8BCH_0(8/3 + 56L/9)$ bytes, and the decimation rows, which set how many members fit on a device; the faster evaluations come from the convolutions and the geometry, the larger ensembles per device from the shared engine and from recomputing the wavelet fields.

Recomputed wavelet fields (preliminary). The reverse sweep can recompute z_n and $|z_n|$ on the convolution rows of each block from the map of the level, as the CPU engine does, instead of reading them from the forward sweep; the forward sweep then holds $|z_n|$ of its current level only, for its decimations, and the reverse chain is applied block by block. A row of z_n is the same whichever block computes it, so kept and recomputed fields give the same statistics, stored moduli and cotangents to the bit for the same blocks (8 fixtures, both forms of the convolution, 1 to 333 blocks per level). The level fields then take $8BCH_0(8/3 + L(8/9 + 5/16))$ bytes at the peak of the reverse sweep, 60 per pixel and member at $L = 4$ (0.75 GB per member at $N_{\rm side} = 1024$, 3.0 GB at 2048), against 188 when they are kept with the per-block chain and 220 with the chain on whole levels; the engine keeps them when they take at most half of the free memory. Recomputing costs one convolution of the maps per block, 0.3 to 0.6 s per evaluation, and the memory it frees goes to larger blocks and a larger geometry cache: on one T4 at $N_{\rm side} = 1024$ (diagonal form, Table 3), recomputing is 2.6% slower with one member and 5.6% faster with four (46.2 against 48.9 s per evaluation, peak 11.9 against 14.5 GB), where the kept fields leave only 1.3 GB of cache. On the A100 at $N_{\rm side} = 2048$, five members take 163 s per evaluation either way, with fields of 15.0 instead of 47.3 GB and a peak of 63.8 instead of 75.1 GB, and ten members now fit: 333 s per evaluation after a setup of 341 s, with a 67.5 GB peak made of the level fields (30.1 GB, 45%), the geometry cache (22.3 GB, 33%: the masks of every block and the coefficients of some), the decimation rows (11.9 GB, 18%) and the transients, topology and CUDA context (3.2 GB). The losses of the two modes agree to 2.7×10^{-15} (different block boundaries). The device is busy 11% of the time: its phases take about 70 s of an evaluation, the rest is host work of the objective and of L-BFGS on ten maps of 5×10^7 pixels with 6 threads, which the next paragraph moves to the device. With few members the decimation rows, about 200 bytes per held cell whatever the number of members, then take more than the fields

(at $N_{\rm side} = 2048$, 11.9 GB of rows against 3.0 GB of fields per member).

The objective on the device (preliminary). The whole objective can be evaluated on the device, not only the sweeps of the statistics: the level power and the kurtosis per level build the pyramid of four children there, the pixel marginal sorts each member with a stable radix sort that reproduces the host's order by value then pixel, and the transform $\kappa = f(g)$ (the monotone cubic interpolation of Sect. 5) and its derivative are computed there. The element-wise expressions are those of the host, in its order, so these terms and their gradients are identical to the bit; the sums over pixels go by chunks of 65 536 cells added in a fixed tree. With several ranks per group, every rank works on its owned slice, the moments of the pyramid are summed over the ranks, and the marginal sorts each member's slice on the device and merges the ranks' orders on the host. On one and three ranks the loss is that of the host path and the gradient agrees with it within 1.2×10^{-16} of its maximum. L-BFGS stays on the host, since its history ($2 \times 10 \times 4$ GB for ten maps at $N_{\rm side} = 2048$) does not fit next to the fields, with its vector operations threaded by chunks summed in a fixed order. During a sweep nothing but the engine's fields is on the device, so the geometry cache keeps its budget. On one T4, ten members at $N_{\rm side} = 1024$ with the terms of the production ensembles (the kurtosis row of Table 6) and recomputed fields take 107.5 s per evaluation instead of 205.1 s with the terms on the host (1.9 times faster), with a 13.6 GB peak (fields 7.5 GB, decimation rows 3.0 GB, cache 2.2 GB) and losses equal to 10^{-15} (4 evaluations, 12 host threads). The marginal goes from 35.1 to 1.2 s per evaluation, the kurtosis, κ transform and level power together from 24.5 to 0.3 s, the copies and transfers from 19.0 to 1.7 s, and L-BFGS from 4.8 to 1.2 s; the device is busy 65% of the run (setup included) instead of 42%, and 95% of the evaluations alone (sampled every 2 s): the sweeps take 100 of the 107 s and saturate the T4. This one 16 GB device is then about as fast as the 640 Irène cores of the same ensemble (128.4 s per evaluation, Sect. 3.5). On three T4s, one rank each, five members at $N_{\rm side} = 2048$ with the same terms take 134.6 s per evaluation instead of 184.7 s with the terms on the host (peak 13.5 GB per device, identical losses), and the devices are busy 69% of the evaluations instead of 50%: with several ranks, the merge of the ranks' orders in the marginal (17 s per evaluation) and the halo exchanges, which copy whole level fields through the host (24 s), still cross the host. A single-rank run has neither.

Table 3. Seconds per evaluation and device memory peak (GB per GPU, cache included) on GPUs ($L = 4$, kernel 5, double precision), with the bound stencils and with the diagonal (DIA) form of the convolution, now the engine's default (Sect. 4); –: not measured. *B*: maps synthesised together.

			Bound stencils		DIA	
$N_{\rm side}$	B	GPUs	s / eval.	mem.	s / eval.	mem.
256	1	1 T4	1.26	9.1	0.51[e]	4.2[e]
256	1	3 T4	0.34	4.0	–	–
1024	1	1 T4	66.8	12.3	22.9[e]	12.0[e]
1024	1	3 T4	13.3	10.9	–	–
2048	1	2 T4	474	13.8	–	–
2048	1	3 T4	114	12.9	–	–
1024	1	1 A100[f]	4.87	62.0	3.04	61.0
2048	1	1 A100[f]	46.3	67.4	17.9	63.2
512	10[a]	2 T4	50.6	13.3	–	–
1024	4	1 T4	–	–	46.2[h]	11.9[h]
1024	10[a]	1 T4	–	–	108[i]	13.6[i]
2048	5[a]	3 T4	–	–	135[i]	13.5[i]
1024	10[a]	1 H200[c]	54.3	98	–	–
1024	10[a]	1 A100[c]	86.6	70.2	–	–
1024	10[a]	1 A100[d]	64.4	62.8	–	–
2048	5[a]	1 H200[c]	157	139	–	–
2048	5[a]	1 A100[d]	395	77.7	163[g]	75.1[g]
2048	10[a]	1 A100[d]	–	–	333[h]	67.5[h]
4096[b]	1	1 H200	∼325	–	–	–
4096[b]	1	2 H200	67.9	124	–	–
4096[b]	1	16 V100	25.2	13.9	–	–

Notes. [a] Ensembles with the κ terms of Sect. 5. [b] Synthesis of the $N_{\rm side} = 2048$ convergence map at twice its resolution ($J = 11$, 19 450 coefficients); the H200 memory is mostly geometry cache, which fills the device. [c] One engine per objective, the geometry cache of each capped (16.8 GB for the H200 at $N_{\rm side} = 1024$, 6.3 GB otherwise); without a cap the setup runs out of device memory. [d] One engine shared by all objectives, no cap. A100 rows with [c] or [d]: 3 evaluations. [e] Against a bound-stencil run of the same series, 1.25 s and 9.8 GB at $N_{\rm side} = 256$, 61.4 s and 13.1 GB at 1024. [e], [f] One map, 4 evaluations from white noise, one run per case, losses of both forms equal to 2×10^{-15}; except at $N_{\rm side} = 256$ the peak is set by the geometry-cache budget, which fills the device in both forms. [g] Preliminary: reverse chain applied per block, wavelet fields held whole (47.3 GB); hpc job 2404157. [h] Preliminary: wavelet fields recomputed per block (Sect. 4); held whole, ten members at $N_{\rm side} = 2048$ do not fit; T4 row: lognormal target, 4 evaluations from white noise; A100 row: job 2404157. [i] Preliminary: fields recomputed and the objective's terms on the device (Sect. 4), 4 evaluations; with the terms on the host, 205 s at $N_{\rm side} = 1024$ and 185 s at 2048. T4: garoupe; H200 and A100 (80 GB): hpc cluster; V100: Irène (4 nodes of 4 V100 16 GB, job 6048940).

5. Synthesising the weak-lensing convergence

5.1. Gaussianised synthesis

The convergence κ at low source redshift is dominated by rare, extended haloes: the map used below has a skewness of 8.4 at $N_{\rm side} = 64$. Synthesising κ directly works poorly, and we synthesise its Gaussian anamorphosis instead, as in FOSCAT (Delouis 2022): the pixel of rank r among N becomes $g = \Phi^{-1}((r + 1/2)/N)$, and a synthesis g is mapped back by matching the target's histogram. Three choices made the synthesis of g reliable:

- $L = 4$ *orientations.* With $L = 2$ the syntheses are elongated east–west (east–west to north–south small-scale power 0.89±0.03 against 1.02±0.05 for the target at $N_{\rm side} = 256$).
- *Coarse to fine.* Each resolution starts from the previous one replicated on the children, plus white noise with the rms of the target's scales missing at the coarser resolution; ensembles carry each member to the next resolution.
- *Power of every pyramid level.* The scattering coefficients leave the finest scale loose: with every family below 0.5σ, a synthesis at $N_{\rm side} = 256$ holds only 58% of the variance of the finest details. Writing d_n for the details of level n (the map at $N_{\rm side} = n$ minus the mean of each group of four children) and V_n for their variance, we add

$$\mathcal{L}_P = \frac{w_P}{B} \sum_m \sum_n \left(\frac{V_n(g_m)}{V_n(t)} - 1 \right)^2 , \tag{9}$$

with its exact gradient; at $w_P = 1$ every level is within 0.3% of the target, at no cost and with the statistics unchanged.

5.2. A Gaussian copula at small scales

Ensembles synthesised this way have every family of the gaussianised statistics at about 1σ per member and every level vari-

ance within 4%, and yet their κ power spectrum is 15–20% low at $\ell = 135$–284 and 26% high at $\ell = 600$–767 at $N_{\rm side} = 256$, the same for every member. A Gaussian random field with exactly the target's spectrum of g, mapped through the same table, has the same deficit above $\ell = 100$: at small scales the syntheses have a Gaussian copula. The κ power there is carried by the extreme tail of g (25 pixels above 4σ and 1062 above 3σ out of 786 432), whose arrangement in extended haloes weighs almost nothing in the gaussianised statistics. Synthesising a less gaussianised field (logarithm or κ itself) removes the deficit but loses the bulk (8 to 107σ off in gaussianised space); correcting the spectrum afterwards breaks the finest level; and the power of each level of κ is met by isolated spikes far in the tail.

5.3. Terms on $\kappa = f(g)$

Three terms, evaluated on $f(g)$ with f the monotone cubic interpolation (Fritsch & Carlson 1980) of the gaussianisation table through 256 knots (linear beyond the ends), fix this:

- the reduced χ^2 of the mean, variance, S_1 and S_2 of κ, as Eq. (7) with its own plan, target and ergodic weights, with a factor $w_\kappa = 0.1$;
- the squared 1-D Wasserstein distance (Villani 2009) between the sorted values of each member and of the target,

$$\mathcal{L}_W = \frac{w_W}{B} \sum_m \frac{1}{N} \sum_p \left(g_{m,(p)} - t_{(p)} \right)^2 , \tag{10}$$

  computed by a distributed sample sort (Frazer & McKellar 1970). Without it the κ term is met by pushing values of g off the Gaussian marginal, which the final histogram matching undoes; with it, $f(g)$ is the map the matching produces;
- the kurtosis of the details of κ at the levels $32 \le n \le N_{\rm side}$,

$$\mathcal{L}_K = \frac{w_K}{B} \sum_m \sum_n \left(\frac{K_n(f(g_m))}{K_n(\kappa_t)} - 1 \right)^2 , \qquad K_n = \frac{\langle d_n^4 \rangle}{\langle d_n^2 \rangle^2}, \tag{11}$$

  since the first two terms make the tails of the details too heavy at intermediate scales (excess kurtosis 3 times the target's at $N_{\rm side} = 64$ in a 4-member ensemble at $N_{\rm side} = 256$).

All gradients are exact (checked against finite differences) and chained through $f'(g)$. The recommended weights for B members are $w_\kappa = 0.1$, $w_W = 1000B$, $w_P = B$, $w_K = 0.1B$ and $w_D = 1$: the terms on the members are sums over the members divided by B, so their weights per member are 1000, 1 and 0.1.

Table 4 summarises the effect at $N_{\rm side} = 256$ (4-member ensembles started from $N_{\rm side} = 128$ members, 200 iterations): the κ terms bring the spectrum from 12.5% to 6.6% rms over $\ell = 31$–767, every family of g staying at 1σ per member; the kurtosis term brings every level within 8% of the target's kurtosis without moving anything else. The κ term costs 1.8 times the evaluation (one more set of wavelet transforms, without S_3/S_4), the kurtosis term 3%.

5.4. A term on the Minkowski functionals of the levels (preliminary)

None of the terms above constrains the topology of the excursion sets, and the Minkowski functionals of the ensembles deviate from the target at the intermediate levels (Sect. 6). A further term holds them level by level:

$$\mathcal{L}_M = \frac{w_M}{B} \sum_m \left\langle \left(\frac{V_{k,i}(\kappa_{m,n}) - V_{k,i}(\kappa_{t,n})}{\sigma_{k,n,i}} \right)^2 \right\rangle_{n,k,i} , \tag{12}$$

Table 4. Effect of the κ terms on 4-member ensembles at $N_{\rm side} = 256$ (from `docs/kappa_power_spectrum.md`).

	none	$+\kappa$, W_2	+ kurtosis
$C_\ell(\kappa)$, rms (max)	12.5% (24%)	6.6% (14%)	6.5% (14%)
S_1–S_4 of g (σ)	0.91–1.00	0.90–1.03	0.88–1.04
Kurtosis, $n = 32, 64$	0.6, 1.1	2.3, 3.0	1.03, 1.01
Kurtosis, $n = 128, 256$	–	0.68, 1.32	1.00, 1.08
Cost / evaluation	1	1.8	1.85

Notes. Columns: no κ term, then the κ terms with the Wasserstein distance, then with the kurtosis term. $C_\ell(\kappa)$: rms and maximum relative deviation of the ensemble's κ spectrum from the target's over $31 \le \ell \le 767$; S_1–S_4: residuals per member in units of σ. Kurtosis ratios: medians over the members of the excess kurtosis of the κ details over the target's (50, 100, 340, 513 at $n = 32, 64, 128, 256$); for the control ensemble, 30 and 114 at $n = 32$ and 64 (not measured at 128 and 256). The kurtosis ratios of the second and third columns come from a rerun of the second on a T4.

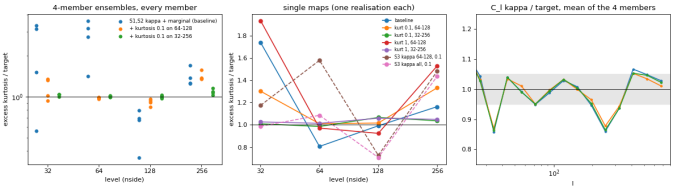


Fig. 3. Kurtosis of the κ details per level over the target's, at $N_{\rm side} = 256$: 4-member ensembles (left) and single maps (middle) with the κ terms, without and with the kurtosis term on two level ranges; right: κ power spectrum over the target's. [TODO: redraw in the paper's style.]

where $\kappa_{m,n}$ is $f(g_m)$ averaged over the 4^n children of each pixel of level n and standardised by its own mean and standard deviation, $V_{k,i}$ ($k = 0, 1, 2$) are its Minkowski functionals at the thresholds $\nu_i = -2, \ldots, 5$, and $\sigma_{k,n,i}$ is the standard deviation of the same estimator over the ten realisations r000–r009 (thresholds where it is below 5% of its maximum are left out). The estimator is made differentiable: the step $\Theta(x - \nu)$ is replaced by a logistic function of width 0.2 and $\delta(x - \nu)$ by its derivative, and the gradient and Hessian of the level map are those of the least-squares quadratic through each pixel and its eight neighbours, fitted in the gnomonic coordinates of the tangent plane. Its gradient, through the stencil, the standardisation and $f'(g)$, is exact (checked against finite differences), and 1 and 4 ranks agree to 10^{-12}. The stencil derivatives see only part of the gradient of a level map: V_1 and V_2 are 0.33–0.49 times the harmonic values of Appendix A, and the deviations of the realisations measured by the two estimators correlate at 0.83–0.92 for V_0 but only 0.15–0.56 for V_1 and V_2. The term therefore cannot be expected to hold every threshold of the harmonic measure.

First tests, judged with the harmonic estimator of Sect. 6, are encouraging. At $N_{\rm side} = 256$ (4 members, levels 64 and 128, $w_M = 0.025$ per member), the largest deviations from r000 drop from 8.4σ (V_1, level 128) and 6.9σ (V_2, level 256) to 3.7 and 5.1σ, the median deviations come back to those of the realisations, and S_1, S_2 of κ improve (from 1.50 and 1.84 to 1.07 and 1.26σ per member) for 10% more time; adding the native level 256 to the term brings the second maximum to 4.4σ. At 0.1 per member the term starts to pull the gaussianised variance of level 128, and at 1 per member it degrades every statistic.

At $N_{\rm side} = 512$ the term was first added to a converged ensemble: the 10-member ensemble with the kurtosis term (450 iterations at $N_{\rm side} = 512$) was continued for 150 iterations without the term and with it at 0.1 and 0.025 per member (levels 64 to

256; Table 5). At both weights the term is met on its own levels (its estimator gives a maximum $|z|$ of 1.6–2.4, against up to 20 for the control), and in the harmonic measure V_0 comes back to the realisations' range at levels 64 to 256 (rms over the thresholds of the members' median z 0.8–1.3, control 1.1–2.9). The offsets of V_1 and V_2 drop at level 128 (rms 2.3 and 2.0 to 1.6–1.7), and those of V_1 at the native level 512, which the term does not include (1.4 to 0.7–0.9). V_1 and V_2 at level 256 and all the maxima do not move (4.2–7.2σ against 4.6–6.2 for the control and 3.5–3.8 for the realisations); since the term's own estimator is within 2.4σ there, what is left is at the pixel scale of each level, which the 3×3 stencil smooths. The power spectrum does not move and the level variances of g change by 1.2σ at most, but the tail of κ pays: the kurtosis of the finest level rises from +1.1 to +3.8 (0.1) and +4.0σ (0.025), the count of peaks above 3σ falls from −0.1 to −1.2 and −0.9σ, and the loss without the term rises from 1.16 to 1.26–1.27. The weight does not set this cost: on the converged ensemble the term starts at 1.6 (0.025) and 6.3 (0.1) times the rest of the loss, and L-BFGS spends the 150 iterations on it in both cases.

The term was then used from the start. The chain that produced this ensemble (200 iterations at $N_{\rm side} = 256$ from the $N_{\rm side} = 128$ members, with the terms S, D and P only, then 450 at $N_{\rm side} = 512$ with the terms of Table 6) was redone from the same members and seed with the term at 0.025 per member at both stages, native level included. At $N_{\rm side} = 256$, V_0 at every level and the medians of V_1 and V_2 at levels 64 and 128 come back to the realisations' range, with maxima of 3.7–5.6σ (control 4.6–7.0); at the native level 256, V_1 and V_2 stay out (maxima 5.6 and 11σ, control 9.7 and 15), and without a kurtosis term at this stage the kurtosis of κ rises to +4.4 and +10.8σ at levels 64 and 128. At $N_{\rm side} = 512$, after the same 450 iterations as the control (last column of Table 5), the kurtosis term of that stage has removed these excursions. V_0 is within the realisations' range at every level, with smaller offsets than when the term is added late (rms 0.9 at level 256, against 1.3, and 2.7 for the control); V_1 at the native level 512 is back within cosmic variance (maximum 2.3σ) and the maximum of V_2 there drops from 7.5 to 4.8σ. The other statistics do not pay for it: the loss without the term is 1.54, against 1.72 for the control at the same count, and S_1–S_4, the level variances of g and the power spectrum are those of the control. What remains is the same as before: the maxima of V_1 and V_2 at levels 64 to 256 stay at 4.2–5.8σ (control 4.4–6.4) while the term's own estimator is within 1.9σ at every level, and the tail of κ at 512 pays as when the term is added late (kurtosis +4.0σ, against +2.3 for the control at 450 and +1.1 at 600 iterations; peaks above 3σ at −0.9σ, control +0.1). An evaluation takes 15% longer than for the control (6.5 h for the 450 iterations on two T4).

The excess kurtosis of the finest level comes from the few most extreme values of g, which the $1/N$ of the Wasserstein term leaves almost free: at $N_{\rm side} = 512$ the three largest values of g of the members lie at 4.77–4.82, against 4.80, 4.90 and 5.11 for the target, and the histogram matching then puts the target's largest values of κ on these pixels, often next to low neighbours. A tail term holds these ranks,

$$\mathcal{L}_T = \frac{w_T}{B} \sum_m \sum_{p \in \mathcal{T}} \left(g_{m,(p)} - t_{(p)} \right)^2 , \tag{13}$$

over the set $\mathcal{T}$ of the K smallest and K largest ranks, without the $1/N$ of Eq. (10). It was added to the chain with the Minkowski term from $N_{\rm side} = 256$, at the $N_{\rm side} = 512$ stage ($w_T = 1$ per member, $K = 100$; last column of Table 5). It holds the extreme ranks and brings the kurtosis of κ at 512 back to +0.9σ (from +4.0), but it degrades most of the rest: the gaussianised variances of levels 128, 256 and 512 go to +8.0, +15.6 and −13.4σ (control +6.0, +4.8, −7.8), power moving from the finest level to the next; the variance of κ at 512 goes to −2.0σ; V_1 and V_2 at levels 256 and 512 move out of the realisations' range (medians 2.0–3.1, maxima up to 10σ); the peaks above 5σ reach +1.2σ; and the largest deviation of the power spectrum goes from 0.6 to 1.5σ. The loss stays three times that of the run without the tail term, and its level-power part does not move over the last 150 iterations. This run cannot tell slow convergence (the curvature of the tail term per pixel is about 3000 times that of the Wasserstein term) from a conflict between the held tail and the variance of the finest level. Against it, the control continued to 600 iterations is as good or better on everything but V_0; against the Minkowski term alone, the control is better on the tail of κ (kurtosis at 512 +1.1 against +4.0σ) and worse on V_0 and on V_1, V_2 at levels 128 and 512.

So far, the Minkowski term is best used from the first stage, at 0.025 per member with the native level: V_0 at every level and V_1 at the native level go back to cosmic variance, the offsets of V_1 and V_2 at the intermediate levels shrink, and the rest of the loss does not suffer; it should not be added to a converged ensemble. It leaves two deviations, whether it starts early or late: maxima of V_1 and V_2 of 4–6σ at levels 64 to 256 (realisations 3.5–3.8), and a kurtosis of κ at the finest level 4σ high. The tail term removes the second at the cost of the gaussianised level variances and of the Minkowski functionals of the finest levels. Neither term is used in the ensembles of Sect. 6 or in the production runs at $N_{\rm side} = 1024$ and 2048, which keep the setting of Table 6; at $N_{\rm side} = 1024$, doubling the iterations of that setting brings V_1 and V_2 at the levels 256 and 512 close to the realisations but leaves V_0 off (Sect. 6.3). Untested: a softer tail term (fewer ranks, a smaller weight) or a longer run with it, a larger kurtosis weight at the finest level, derivatives that keep the pixel scale (a wider stencil or harmonic derivatives), and either term at $N_{\rm side} = 1024$ and 2048.

6. Ensembles of syntheses of the Takahashi convergence

6.1. Data and protocol

The target is the convergence map r000 of the full-sky ray-tracing simulations of Takahashi et al. (2017) for the first source plane ($z_s \simeq 0.05$, `zs1`), given at $N_{\rm side} = 4096$ and degraded to NESTED maps at $N_{\rm side} = 256$ to 2048 by averaging sub-pixels. The nine realisations r001 to r009 of the same suite, with independent initial conditions, measure the cosmic variance of every statistic: none of them is used in the synthesis. An ensemble *matches* the target when its members deviate from r000 by about as much as the realisations deviate from each other.

The ensembles are built from coarse to fine ($N_{\rm side} = 128$, 256, 512, 1024, then 2048), each resolution started from the members of the previous one, with the loss of Eqs. (7), (8) and the terms of Sect. 5 (the $N_{\rm side} = 256$ stage of the $N_{\rm side} = 512$ ensemble with the kurtosis term has S, D and P only, Sect. 5.4), $L = 4$, $K = 5$, $J = \log_2 N_{\rm side} - 1$, and a few hundred L-BFGS iterations per step. Table 6 lists the runs. Each member's g is mapped back to κ by histogram matching with r000.

Table 5. Minkowski functionals of the levels of κ at $N_{\rm side} = 512$ (harmonic estimator) with and without the Minkowski and tail terms: median / maximum $|z|$ over the members and the thresholds, z the deviation from r000 in units of the standard deviation of the realisations. Below: the loss without these terms, then deviations from r000 in units of the realisations' standard deviation (median over the members) of the kurtosis and variance of κ and of the gaussianised variance at the finest levels, of the peak counts and of the power spectrum. Preliminary.

Level		r001–r009	control		$+\mathcal{L}_M$ after 450 it.		$+\mathcal{L}_M$ from $N_{\rm side} = 256$	
			450 it.	600 it.	$w_M = 0.1$	$w_M = 0.025$	alone	$+\mathcal{L}_T$
64	V_0	0.9 / 2.9	0.8 / 3.1	0.9 / 2.9	0.4 / 3.2	0.5 / 3.0	0.4 / 3.0	0.6 / 3.8
	V_1	1.1 / 3.5	1.4 / 5.2	1.3 / 5.3	1.2 / 5.2	1.3 / 5.0	1.3 / 4.6	1.2 / 3.9
	V_2	1.1 / 3.5	1.0 / 4.4	0.9 / 4.6	1.1 / 5.3	1.0 / 4.2	1.1 / 5.6	1.0 / 5.3
128	V_0	1.5 / 3.6	1.5 / 3.3	1.7 / 3.5	0.7 / 2.8	0.7 / 2.9	0.9 / 2.3	1.0 / 3.0
	V_1	1.1 / 3.6	2.0 / 6.1	1.9 / 6.2	1.2 / 5.1	1.3 / 5.4	1.3 / 4.9	1.5 / 5.2
	V_2	0.9 / 3.6	1.7 / 5.8	1.7 / 5.4	1.3 / 5.2	1.3 / 5.4	1.4 / 5.7	1.4 / 4.8
256	V_0	1.9 / 3.6	2.8 / 3.6	3.1 / 3.9	0.7 / 2.0	1.2 / 2.3	0.7 / 1.8	1.4 / 3.5
	V_1	1.2 / 3.8	2.0 / 5.4	1.6 / 5.3	1.9 / 4.9	1.9 / 4.9	1.7 / 4.2	2.8 / 5.7
	V_2	1.1 / 3.7	2.4 / 6.4	2.1 / 5.4	1.9 / 7.2	1.8 / 6.7	1.7 / 5.8	3.1 / 10.1
512	V_1	1.2 / 3.7	1.5 / 3.5	1.3 / 3.1	0.6 / 2.6	0.7 / 2.0	0.9 / 2.3	3.0 / 4.8
	V_2	0.9 / 3.5	1.6 / 7.5	1.5 / 6.7	0.8 / 5.3	0.8 / 5.1	0.8 / 4.8	2.0 / 8.5
Loss without $\mathcal{L}_M$, $\mathcal{L}_T$			1.72	1.16	1.27	1.26	1.54	–
Kurtosis of κ at 512			+2.3	+1.1	+3.8	+4.0	+4.0	+0.9
Variance of κ at 512			+0.4	+0.8	−1.3	−1.0	−1.2	−2.0
Variance of g at 256 / 512			+4.8 / −7.8	+2.6 / −6.4	+3.8 / −7.1	+3.7 / −7.1	+4.1 / −7.4	+15.6 / −13.4
Peaks above 3σ / 5σ			+0.1 / +0.1	−0.1 / 0.0	−1.2 / −0.7	−0.9 / −1.0	−0.9 / −0.5	+0.8 / +1.2
C_ℓ, $\ell \geq 31$: maximum $\lvert z\rvert$			0.6	0.6	0.6	0.5	0.6	1.5
Time per evaluation (s, 2 T4)			41.3	40.8	43.2	42.8	47.3	48.2

Notes. Iterations at $N_{\rm side} = 512$. Control: the ensemble with the kurtosis term of Table 6 (450 iterations) and its continuation by 150 iterations. $+\mathcal{L}_M$ after 450 it.: the same continuation with the Minkowski term on levels 64 to 256. $+\mathcal{L}_M$ from $N_{\rm side} = 256$: the whole chain redone with the term ($w_M = 0.025$), native level included (levels 64 to 256 at $N_{\rm side} = 256$, 64 to 512 at $N_{\rm side} = 512$), 450 iterations at $N_{\rm side} = 512$, alone or with the tail term at the $N_{\rm side} = 512$ stage ($w_T = 1$, $K = 100$). Weights per member. At level 512 the members have r000's histogram, so V_0 (median $|z|$ below 0.7 in every run) tests only the reconstruction and is left out. Times with the bound stencils (Sect. 4).

Table 6. Ensembles of 10 syntheses used in Sect. 6.

$N_{\rm side}$	Label	Loss terms	Iterations	s / eval.	Hardware
512	no kurtosis	S, D, P, κ, W	300	50.6	2 T4 (2 groups)
512	kurtosis	S, D, P, κ, W, K	300 + 150	41.3	2 T4 (2 groups)
1024	no κ terms	S, D, P ($w_P = 1$)	300	149	160 EPYC 9655 cores (10 groups)
1024	κ terms	S, D, P, κ, W	300	54.3	1 H200
1024	κ terms, kurtosis[a]	S, D, P, κ, W, K	300 + 300	128 / 258	640 Rome / 448 Xeon cores (10 groups)
2048	[TODO: kurtosis, Datarmor[b] (running)]	S, D, P, κ, W, K	100 + 100	~1070	448 Xeon cores [TODO: groups]

Notes. S: scattering covariance of the ensemble mean (Eq. 7); D: scatter (Eq. 8, $w_D = 1$); P: power per level ($w_P = 10$ unless stated); κ: mean, variance, S_1, S_2 of κ ($w_\kappa = 0.1$); W: Wasserstein ($w_W = 10^4$); K: kurtosis ($w_K = 1$, levels 32 to $N_{\rm side}$). [a] 300 iterations on Irène, then 300 on Datarmor (400 ranks in 10 groups of 40 over 448 Xeon E5-2680 v4 cores, Intel MPI); analysed separately (preliminary, end of Sect. 6.3); the figures and tables of Sect. 6.3 show the two other $N_{\rm side} = 1024$ ensembles. [b] From the members of [a] after 600 iterations. [TODO: 2048 settings from the production chain.] GPU rows: bound stencils (Sect. 4).

6.2. Cosmic variance

Over the ten realisations, the relative standard deviation is 8–18% for the variance of the κ details per level, 13–35% for their excess kurtosis, 7–17% for the power spectrum in logarithmic bands above $\ell = 30$, 6–9% for the peak counts above 3, 5 and 10σ, and only 1–4% for the variance of the details in gaussianised space: the rank transform removes the halo-dominated tail, and what remains is a much more precise statistic. r000 is a typical realisation, within 1.9σ of the mean of the ten for every statistic (Table 7).

6.3. Results

Figures 4 and 5 show the target, another realisation, a Gaussian field with the same power spectrum and members of the ensemble; Figs. 6–9 and Table 7 compare the statistics. We quote the $N_{\rm side} = 1024$ ensembles below.

Power spectrum. Without the κ terms the members' spectrum is 24% rms off r000 over $\ell \geq 31$, up to 8σ in the last band. With them, it is within 5.9% rms and within 1σ of r000 in every band (median $|z| = 0.24$; Fig. 6). At $N_{\rm side} = 512$ both ensembles, which carry the κ terms, are within 3.4–4.4% rms and 0.6σ.

Per-level statistics (Fig. 7). The variance of the κ details is within 1.1σ of r000 at every level with the κ terms, against $+2.5\sigma$ and -2.1σ at the levels 256 and 1024 without. The excess kurtosis overshoots at intermediate levels with the κ terms and no kurtosis term ($+10.7\sigma$ in median at level 64); at $N_{\rm side} = 512$ the kurtosis term brings every level from 32 to 256 within 1σ (level 64 from $+5.3$ to 0.0σ), the finest level remaining at $+2.3\sigma$, and at $N_{\rm side} = 1024$ every level up to 512 after 300 iterations and from 16 to 1024 after 600 (the ensemble with the kurtosis term, below). The variance in gaussianised space, the most precise statistic, is where the ensembles are furthest from the target: +12 and $+14\sigma$ (+16 and +22%) at the levels 128 and 256, and -8.7σ (−31%) at the finest level for $N_{\rm side} = 1024$.

Peaks and PDF (Fig. 8). The counts of peaks above 3, 5 and 10σ are within 1σ with the κ terms, against $+4\sigma$ (above 3σ) without. The histograms of peak heights at $N_{\rm side}/4$ show the κ-term ensemble with 20–30% too many peaks between 3 and 10σ at that resolution.

Minkowski functionals (Fig. 9, Appendix A) are the most demanding test: none of the terms of the loss constrains them directly. The members deviate from r000 by 2.1–2.5σ rms over the thresholds $-2 \leq \nu \leq 5$ for the map smoothed over 4 pixels (13.7 arcmin), and by 1.1–2.5σ for a 30 arcmin smoothing, with the largest deviations at $\nu \simeq 1$–3 for V_1 and V_2, i.e. in the connectivity of the moderately high excursion sets. Per pyramid level (Table 8 and Fig. 10: maps degraded by 4-children means, no smoothing), the reference is the deviation of the realizations r001–r009 from r000, a median $|z|$ of 0.8 to 2.0 and a maximum of 3.2 to 3.8 (r000 itself sits 1.5–2σ from the mean of the realizations in V_0 at the levels 128 to 512). At level 64 the members' median $|z|$ (0.9–1.5) is that of the realizations, apart from a few thresholds in the low tail. At the levels 256 and 512 it is 3.0–3.9, with systematic offsets of 3–7σ in V_1 and V_2 for $0 \lesssim \nu \lesssim 3$, the largest deviations being at $\nu \lesssim 0$ (the troughs). At the native level the members are tens to hundreds of σ off, the trace of the deficit of the finest gaussianised level. A loss term on these functionals (Sect. 5.4), used from the $N_{\rm side} = 256$ stage on, brings V_0 at every level and V_1 at the native level back to the realisations' range at $N_{\rm side} = 512$ without raising the rest of the loss, but leaves the largest deviations of V_1, V_2 at the intermediate levels, and the kurtosis of the finest level rises to $+4\sigma$; a further term on the extreme ranks of g brings the kurtosis back but degrades the gaussianised level variances and the Minkowski functionals of the finest levels (preliminary).

The ensemble with the kurtosis term at $N_{\rm side} = 1024$ (Table 6: 300 iterations on Irène, loss 2.46, then 300 more with the same setting on Datarmor, loss 0.80; preliminary until the $N_{\rm side} = 2048$ analysis). Measured with the same scripts against r000–r009, after 300 iterations the variance of the κ details is within 1σ of r000 at every level from 8 to 1024 (worst -0.8σ at 256), and their excess kurtosis is within 1σ up to level 512 and at $+1.3\sigma$ at 1024, where the ensemble without the kurtosis term overshoots by +10.7, +3.3 and $+3.6\sigma$ at the levels 64, 128 and 1024. The power spectrum stays within the cosmic variance (median $|z| = 0.3$, maximum 1.2 over the bands $\ell \geq 31$), and the counts of peaks above 3, 5 and 10σ are at +1.1, +1.0 and $+0.1\sigma$. The gaussianised variances of the levels 128 and 256 are closer to the target than without the kurtosis term, but all remain outside the cosmic variance: $+8.6$ and $+9.8\sigma$ (+11 and +16%) at the levels 128 and 256 (against +12 and $+14\sigma$), -9.4σ (-34%) at the finest level. The Minkowski functionals of the levels do not improve: the median $|z|$ is 0.9–1.2 at level 64, as for the realisations, but 1.7–2.2 at 128, 2.2–4.1 at 256 and 2.4–4.2 at 512 (realisations 0.9–2.1), with maxima of 6–9 in V_1 and V_2, as without the kurtosis term. Doubling the iterations removes most of these discrepancies. After 600 iterations, the gaussianised variances are at $+3.9\sigma$ (+5%) at level 128 and at -1.2, -2.1 and -2.5σ (-2, -5 and -9%) at the levels 256, 512 and 1024, the levels 32 and 64 staying at +2.4 and $+3.3\sigma$. The excess kurtosis of κ is within 0.3σ at every level from 16 to 1024 (level 8 at -1.7σ, as after 300 iterations). V_1 and V_2 at the levels 256 and 512 come close to the realisations, with a median $|z|$ of 1.3–1.9 and maxima of 4.0–6.0, against 2.2–4.2 and 6.5–9.1 after 300 iterations. The power spectrum does not move (maximum $|z|$ 1.3), and the peaks above 3, 5 and 10σ are at +1.2, +0.6 and -0.7σ. Two statistics do not follow. The variance of the κ details at the finest level rises to $+2.9\sigma$ (members from +2.3 to +3.4; about 27% above r000) and falls to -1.8σ at level 256. V_0 stays off at the levels 256 and 512 (median $|z|$ 4.1 and 3.1, against 1.9 and 2.1 for the realisations), and the levels 64 and 128 hardly change. Holding the kurtosis of the levels does not by itself hold the topology of their excursion sets, but much of the deviation after 300 iterations was unfinished convergence; what remains in V_0 is what the term of Sect. 5.4 brought back at $N_{\rm side} = 512$.

Table 7. Deviation of the ensemble members from the target r000 at nside 1024, in units of the standard deviation σ of the 10 Takahashi realizations: median [min, max] over the members (bold: $|z| > 2$). σ/μ is the realizations' relative standard deviation and the r000 column the target's own deviation from their mean. Details of level n: the map at nside n minus the mean of each group of 4 children. The C_ℓ and Minkowski rows give the rms over the bands ($\ell \geq 31$) or thresholds ($-2 \leq \nu \leq 5$, FWHM 13.74′) of each member's deviation.

Statistic	Level	σ/μ	r000	no kappa terms	kappa terms
variance, κ					
	8	13.4%	+0.1	−0.7 [−1.1, +1.8]	−0.3 [−0.8, +0.6]
	16	18.0%	−0.2	−0.2 [−0.4, +1.8]	+0.2 [−0.3, +0.6]
	32	15.0%	−0.1	−0.5 [−1.0, +1.4]	−0.0 [−0.6, +0.9]
	64	14.7%	−0.1	−0.2 [−0.4, +0.4]	+0.4 [−0.2, +0.9]
	128	13.0%	+0.2	+0.7 [+0.2, +1.6]	+0.0 [−0.5, +1.1]
	256	11.7%	+0.1	**+2.5** [**+2.2**, **+2.7**]	−0.0 [−0.5, +0.2]
	512	10.9%	−0.1	−0.9 [−1.9, −0.5]	−0.2 [−0.4, −0.0]
	1024	9.3%	−0.0	**−2.1** [**−3.3**, −1.7]	−0.1 [−0.6, +0.4]
excess kurtosis, κ					
	8	31.4%	+0.7	**−2.3** [**−3.0**, **+32.8**]	−1.3 [−1.7, **+3.4**]
	16	35.2%	+1.4	−1.7 [**−2.3**, **+61.8**]	**+2.7** [−1.7, **+5.5**]
	32	35.3%	+1.1	**−2.0** [**−2.7**, **+23.1**]	**+2.9** [−1.0, **+10.8**]
	64	27.9%	+0.7	+0.0 [−1.7, **+6.5**]	**+10.7** [**+4.5**, **+35.3**]
	128	29.8%	+1.9	−0.8 [**−2.3**, **+11.7**]	**+3.3** [−1.5, **+10.7**]
	256	21.3%	+1.6	**+4.8** [**+2.5**, **+9.6**]	+1.7 [−1.9, **+4.8**]
	512	16.3%	+1.4	**+2.8** [−1.4, **+6.5**]	−0.0 [−1.5, **+2.6**]
	1024	17.3%	+1.1	**+2.7** [**−2.4**, **+5.7**]	**+3.6** [+1.2, **+10.6**]
variance, gaussianised					
	8	5.6%	+0.6	−0.2 [−0.6, +0.3]	−0.1 [−0.3, +0.2]
	16	3.7%	−0.7	−0.1 [−0.5, +0.5]	+0.1 [−0.1, +0.4]
	32	2.2%	−1.1	+0.6 [+0.2, +1.1]	+1.2 [+0.9, +1.6]
	64	2.0%	−1.3	**+3.0** [**+2.9**, **+3.4**]	**+4.0** [**+3.9**, **+4.3**]
	128	1.3%	+0.2	**+12.6** [**+12.2**, **+12.7**]	**+12.0** [**+11.8**, **+12.2**]
	256	1.6%	+1.1	**+22.4** [**+22.2**, **+22.6**]	**+13.9** [**+13.7**, **+14.1**]
	512	2.4%	+1.3	+0.2 [+0.1, +0.4]	+0.7 [+0.6, +0.9]
	1024	3.6%	+1.1	**−11.1** [**−11.1**, **−11.0**]	**−8.7** [**−8.8**, **−8.5**]
peaks					
	$> 3\sigma$	5.8%	−0.4	**+4.0** [**+3.8**, **+4.1**]	−0.2 [−0.4, −0.1]
	$> 5\sigma$	6.6%	−0.3	**+3.1** [**+2.8**, **+3.3**]	+0.3 [+0.1, +0.5]
	$> 10\sigma$	8.1%	−0.2	+1.7 [+1.3, **+2.1**]	+0.8 [+0.6, +1.1]
C_ℓ					
	rms, $\ell \geq 31$			2.6 [2.3, 2.8]	0.6 [0.5, 1.0]
Minkowski					
	V_0, rms			1.8 [1.4, 3.2]	2.3 [2.0, 2.4]
	V_1, rms			1.3 [1.1, 3.7]	2.2 [1.8, 2.7]
	V_2, rms			3.5 [3.0, 5.4]	2.5 [2.4, 2.7]

Table 8. Minkowski functionals of κ degraded to each level (nside n, means of 4 children, no smoothing), thresholds $-2 \leq \nu \leq 5$: deviation from r000 in units of the standard deviation of the 10 realizations. Each cell: median $|z|$ and maximum $|z|$ over the members and the thresholds, and rms over the thresholds of the members' median z. The first column applies the same measure to the realizations r001–r009, the reference for an ensemble within cosmic variance.

Level		r001–r009	kappa terms	no kappa terms
64	V_0	1.0 / 3.4 / 1.1	1.5 / 14.6 / 3.0	1.8 / 10.8 / 1.9
64	V_1	1.1 / 3.4 / 1.1	1.1 / 15.3 / 1.2	1.4 / 9.0 / 1.6
64	V_2	1.1 / 3.5 / 1.1	0.9 / 14.7 / 1.2	1.1 / 9.5 / 1.5
128	V_0	1.6 / 3.6 / 1.7	2.2 / 22.0 / 3.1	2.4 / 26.2 / 5.7
128	V_1	1.2 / 3.6 / 1.2	1.9 / 14.1 / 2.2	2.7 / 16.9 / 4.3
128	V_2	1.0 / 3.6 / 1.1	1.6 / 11.2 / 2.0	2.0 / 11.2 / 2.9
256	V_0	2.0 / 3.6 / 2.0	3.0 / 11.0 / 3.6	1.6 / 76.3 / 21.5
256	V_1	1.2 / 3.8 / 1.3	3.0 / 9.8 / 3.1	2.7 / 54.9 / 14.4
256	V_2	1.2 / 3.7 / 1.2	3.4 / 13.2 / 3.7	2.6 / 38.6 / 7.6
512	V_0	2.0 / 3.6 / 2.0	3.1 / 35.1 / 7.1	1.4 / 353.3 / 75.0
512	V_1	1.1 / 3.7 / 1.2	3.9 / 21.5 / 5.3	2.0 / 157.8 / 35.8
512	V_2	0.9 / 3.4 / 0.9	3.5 / 24.3 / 5.9	1.5 / 90.3 / 21.3
1024	V_0	1.7 / 3.5 / 1.8	0.5 / 571.1 / 67.4	1.8 / 1235.2 / 316.8
1024	V_1	1.1 / 3.5 / 1.1	4.2 / 1174.1 / 89.1	7.7 / 3489.1 / 513.1
1024	V_2	0.8 / 3.2 / 0.8	3.1 / 129.0 / 26.3	2.4 / 578.2 / 133.5

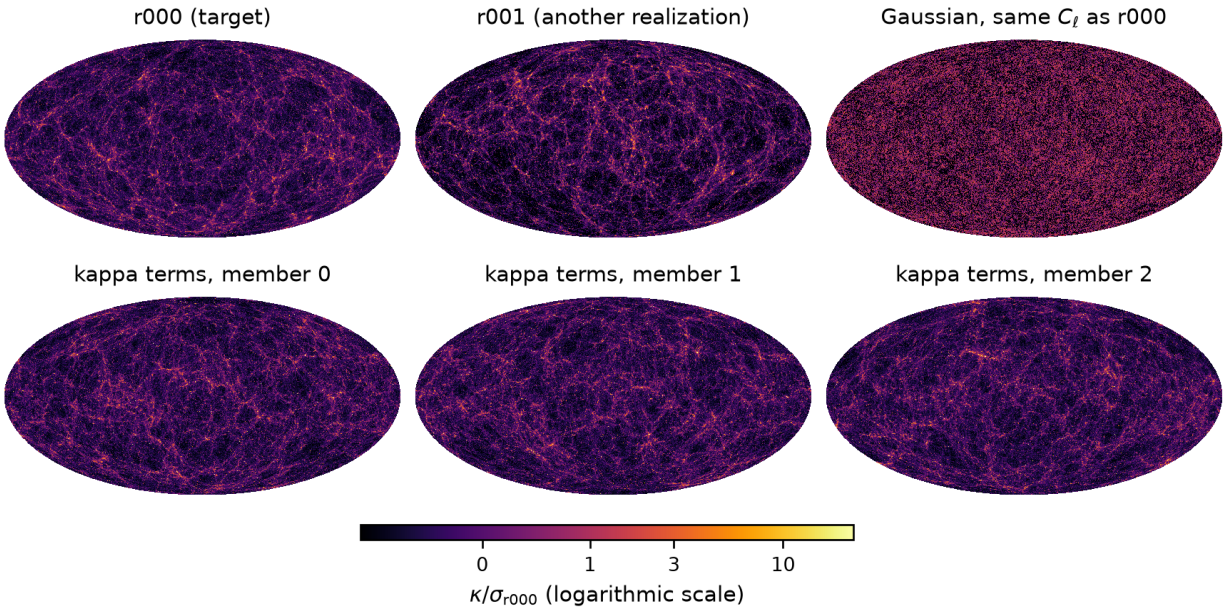


Fig. 4. Convergence maps at $N_{\rm side} = 1024$ (Mollweide). Top: the target r000, an independent realisation r001 of the same simulation suite, and a Gaussian field with the angular power spectrum and the mean of r000; bottom: three members of the ensemble with the κ terms. As in Mousset et al. (2024), the colour is logarithmic so that the texture shows: $u = \kappa/\sigma_{\rm r000}$ is displayed as $\log_{10}(u - u_{\rm min} + 0.1)$, with $u_{\rm min} = -0.41$ the minimum of r000, on the same scale in every panel (the colour bar is labelled in u). The Gaussian field takes values below $u_{\rm min}$, which κ cannot, shown at the bottom of the scale.

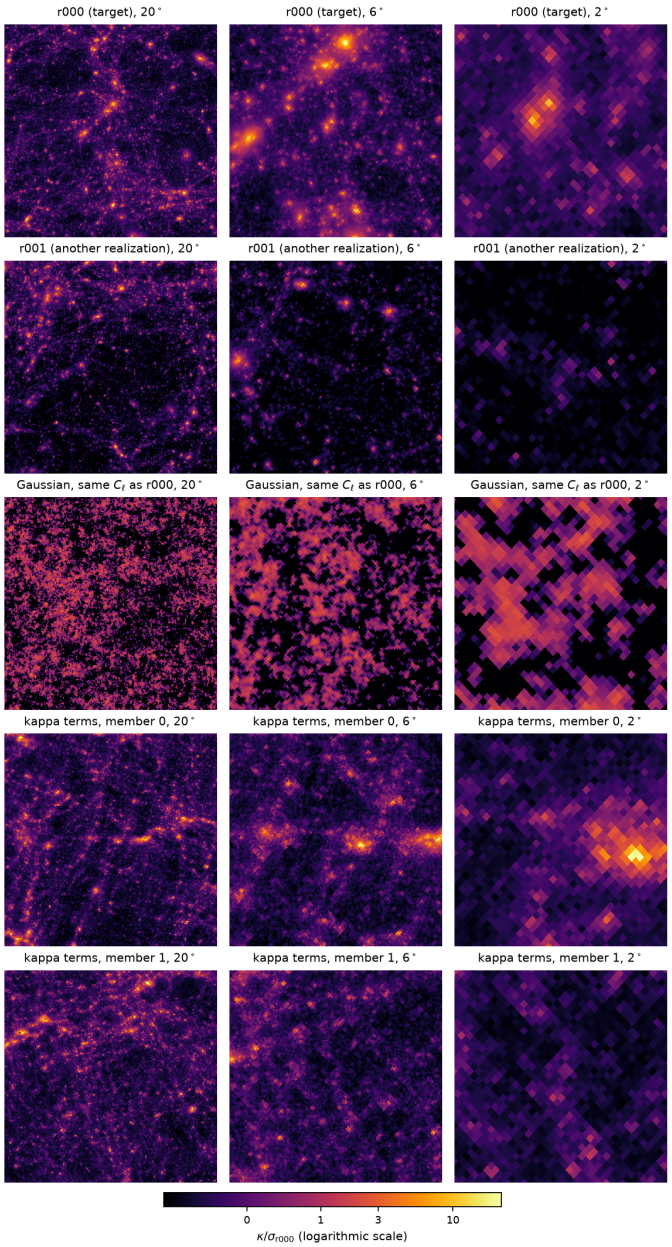


Fig. 5. Gnomonic zooms of 20, 6 and 2 degrees on the same centre in the maps of Fig. 4 (rows: r000, r001, the Gaussian field, two members), with the same logarithmic colour scale.

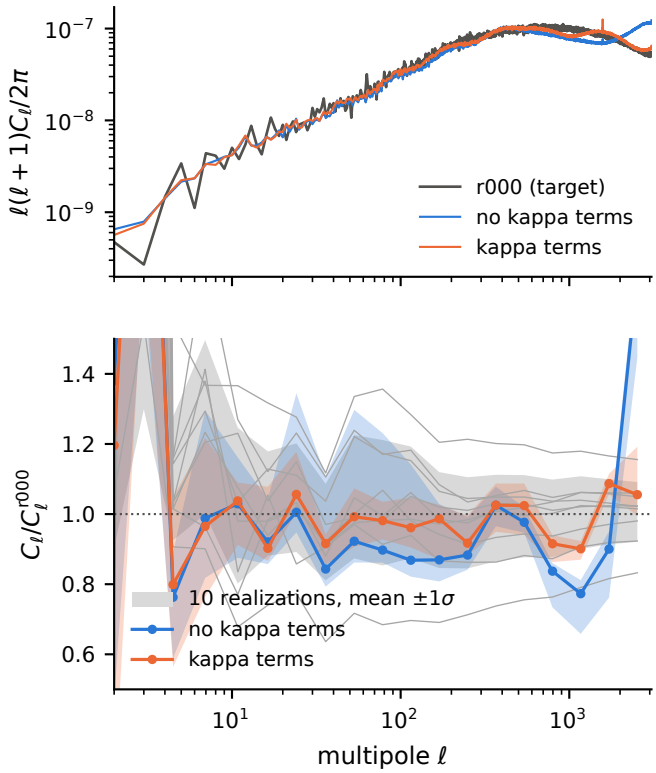


Fig. 6. Power spectrum of κ at $N_{\rm side} = 1024$. Top: $\ell(\ell+1)C_\ell/2\pi$ of r000 and of the members (median). Bottom: ratio to r000 in logarithmic bands; grey: mean $\pm1\sigma$ of the 10 realisations, thin lines r001–r009; colours: median and range of the members.

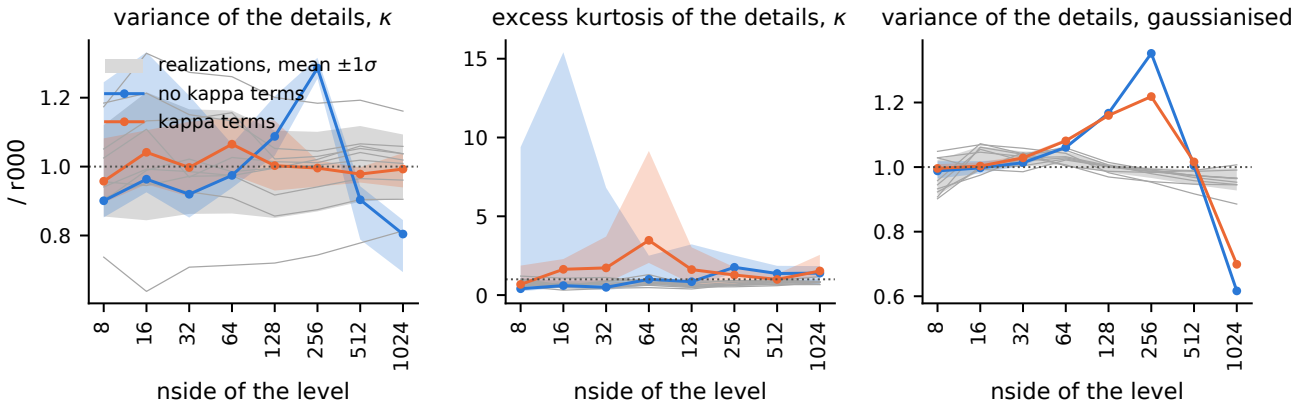


Fig. 7. Statistics of the 4-children pyramid details per level, over r000's: variance and excess kurtosis of κ, variance in gaussianised space (each map's own rank transform). Grey: realisations, mean $\pm 1\sigma$ and individual curves; colours: median and range of the members.

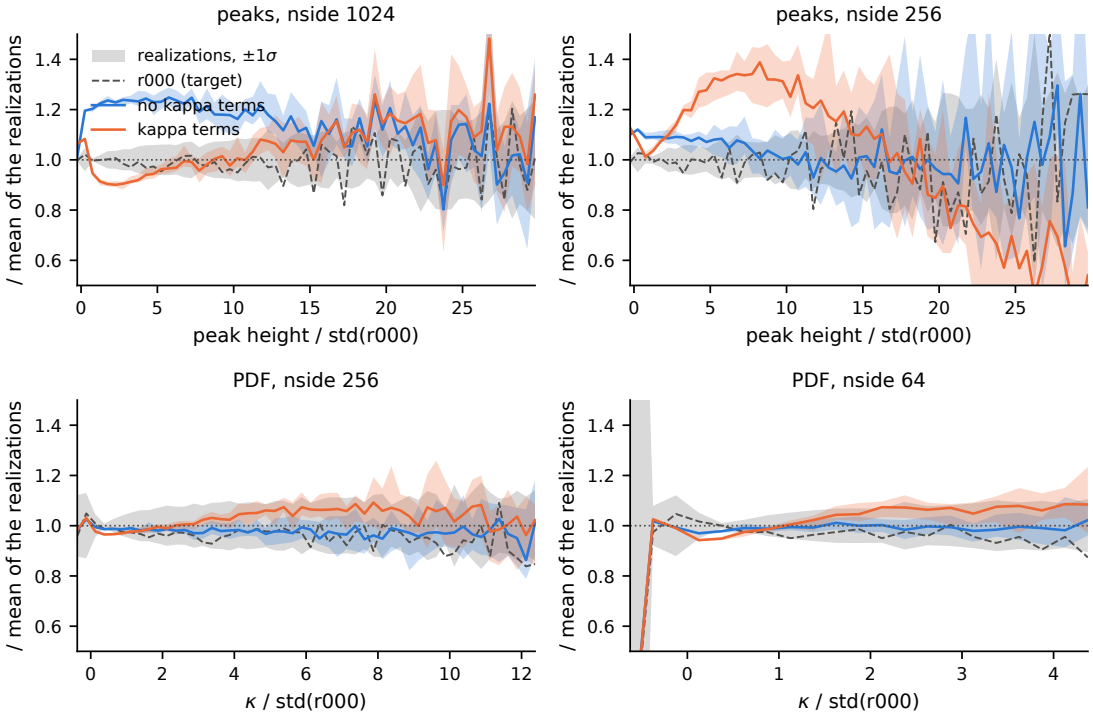


Fig. 8. Top: number of peaks (pixels higher than their 8 neighbours) per bin of height, at $N_{\rm side}$ and $N_{\rm side}/4$; bottom: one-point PDF of κ degraded to $N_{\rm side}/4$ and $N_{\rm side}/16$ (at full resolution the members have r000's histogram by construction). All divided by the mean of the 10 realisations.

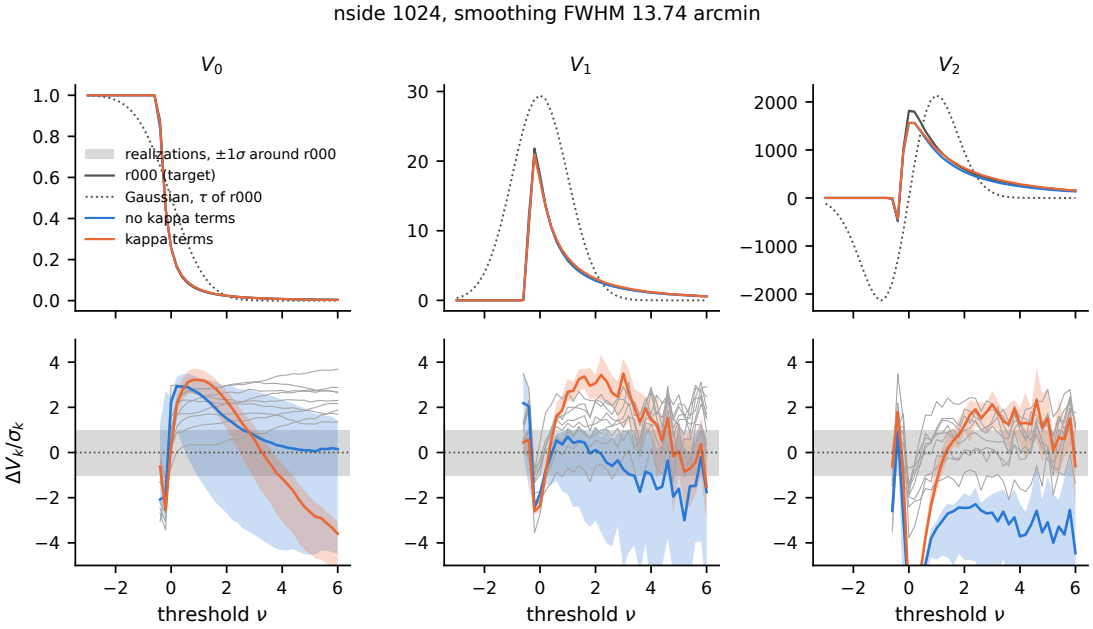


Fig. 9. Minkowski functionals of κ smoothed over 4 pixels. Top: V_0, V_1, V_2 of r000, of the members (median) and the Gaussian expectation for r000's spectral moment; bottom: deviation from r000 in units of the realisations' standard deviation. The 30 arcmin smoothing is in `figures/minkowski_n1024_s1.pdf`.

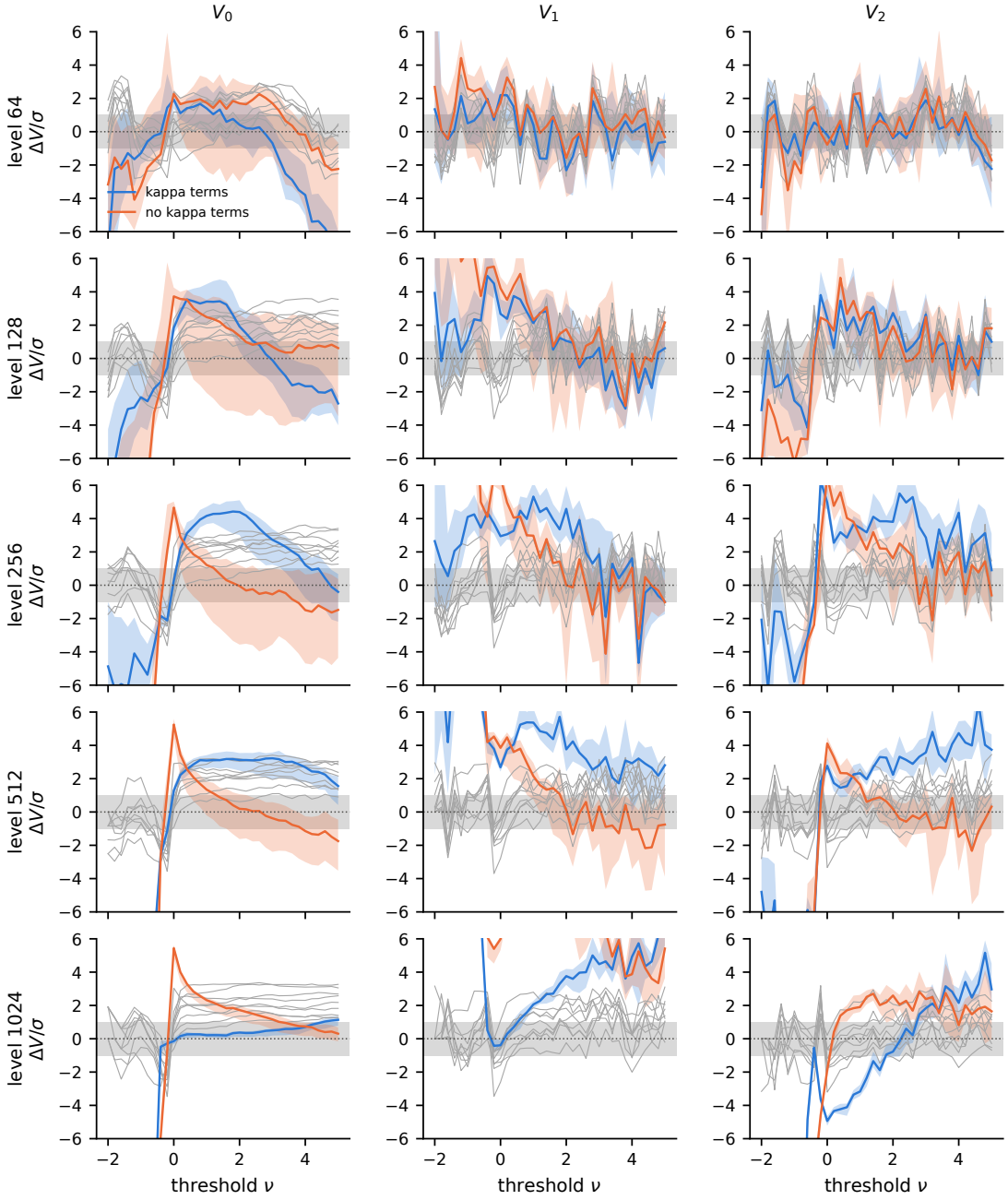


Fig. 10. Minkowski functionals of κ degraded to each level of the pyramid (means of 4 children, no smoothing): deviation from r000 in units of the standard deviation of the 10 realizations. Grey: r001–r009; colours: median and range of the members.

7. Discussion and outlook

7.1. What the distributed engine changes

The explicit, block-wise gradient turns the memory of a scattering-covariance synthesis from a property of the device into a property of the machine. The state of a synthesis is a few hundred bytes per pixel and per map, divided by the number of processes, and the transient memory is set by the block size, not by the map. On CPUs the time per evaluation drops with the number of ranks as long as a rank owns at least about 10^4 pixels of level 0; on GPUs the limit is the device memory, which the level fields (60 bytes per pixel and map at $L = 4$ when recomputed per block) and the decimation rows (about 200 bytes per held cell) set once the convolution geometry is held in diagonal form (16 bytes per row between sweeps), and several devices per synthesis lift it. An ensemble of ten $N_{\rm side} = 2048$ maps, 5×10^8 unknowns, is a routine job on a few hundred CPU cores or a few GPUs. The determinism of the implementation (identical losses for any rank count) makes such runs reproducible and comparable across machines.

7.2. Limits of the present syntheses

The weak-lensing ensembles show two limits, the optimiser's and the statistical model's. After 300 iterations the gaussianised level variances are 10–30% off at a few levels, far outside a cosmic variance of 1–4%, and the Minkowski functionals, which probe the topology of the excursion sets, deviate by 2–3σ. At $N_{\rm side} = 1024$, 300 more iterations bring the former to 2–9% (1–4σ) and V_1, V_2 of the finest levels close to the realisations, so part of these deviations is unfinished convergence (preliminary). What remains points to the model: the spectrum of g oscillates from one octave to the next by about 10%, because dyadic wavelet bands fix the power of each octave but not its shape within it; V_0 stays off at the finest levels, and the variance of the finest κ details ends 27% high; a first loss term on the Minkowski functionals, used from the $N_{\rm side} = 256$ stage on, brings V_0 and the native level back at $N_{\rm side} = 512$ without raising the rest of the loss, but leaves maxima of 4–6σ at the intermediate levels, because its stencil derivatives see only part of what the harmonic measure sees, and the kurtosis of the finest level rises to about +4σ; holding the extreme ranks of g brings it back but shifts power between the two finest levels (Sect. 5.4). The per-level kurtosis, held by the kurtosis term, keeps little scatter between the members (a few tenths of σ at the intermediate levels at $N_{\rm side} = 1024$), and holding it does not hold the Minkowski functionals. These point to statistics to add (finer frequency sampling, cross-level terms on κ) as well as to longer runs. [TODO: update with the 2048 ensemble.]

7.3. Towards climate and ocean data

The fields of Earth-system science are larger than the cosmological maps of this paper in every direction: higher resolution (global kilometre-scale models and reanalyses), many channels (temperature, salinity, velocities, sea-surface height, at several depths or pressure levels) and time. The engine already handles C channels and their cross-covariances, and its memory per process is set by the share of the sky, so the first two directions are a matter of node count. A global ocean model at 1/12° has about 10^7 surface cells, comparable to $N_{\rm side} = 1024$; a kilometre-scale atmosphere on HEALPix at $N_{\rm side} = 8192$ to 16 384 has 0.8 to 3.2×10^9 cells per level, which at about 400 bytes per pixel and map (engine and L-BFGS, $L = 4$) needs 3 to 13 TB of state for an ensemble of ten: the memory of 5 to 20 nodes of the hpc cluster used here. Time is the larger cost: extrapolating the Irène rate linearly in the number of pixels (12.8 s per evaluation for 1.3×10^7 pixels on 512 cores, $L = 4$), one evaluation of a single $N_{\rm side} = 8192$ map would take about 14 minutes on 512 cores, or about a minute on 7×10^3 cores. Three extensions are needed: masks and missing data (land for the ocean, survey footprints in cosmology), which the block decomposition accommodates by dropping blocks and weighting halos; time, as an extra dimension of the wavelets or as cross-covariances between consecutive fields, which multiplies the channels but not the halos; and lower-precision arithmetic on GPUs, which would require validating the statistics in mixed precision. Such emulators would complement the large probabilistic forecasting models (Price et al. 2025): they need no training archive, they produce realisations constrained by interpretable statistics, and they extend to the resolution of the latest models at the cost of more nodes, not of a new network.

8. Conclusions

We have presented STL-Dev-MPI, a distributed implementation of scattering-covariance synthesis on HEALPix. Its explicit pyramidal, block-wise gradient bounds the memory of each process by its share of the sky; its MPI decomposition, with point-to-point halos, ordered ghost reductions and a distributed L-BFGS, scales with 75–85% efficiency to hundreds of cores and gives identical results for any rank count; its GPU engine, with one engine per device shared by all the objectives and wavelet fields recomputed per block, runs ten $N_{\rm side} = 2048$ syntheses on one 80 GB A100, ten at $N_{\rm side} = 1024$ on one 16 GB T4 about as fast as 640 CPU cores when the objective's terms also run on the device (both preliminary), and an $N_{\rm side} = 4096$ synthesis on two H200s, and the diagonal form of its convolution makes an evaluation 1.6 to 2.7 times faster with a third of the geometry memory. Applied to the weak-lensing convergence, with terms on the statistics of κ, on the pixel histogram, on the power and kurtosis of each pyramid level and on the scatter of the ensemble, it produces ensembles whose power spectrum, per-level variance and kurtosis, and peak counts lie within the cosmic variance of independent realisations, while the gaussianised level variances and the Minkowski functionals show what the statistical model still misses. [TODO: conclusions of the 2048 ensemble.] The code, the fixtures and the scripts of every figure are in the STL-Dev-MPI repository, to be made public with the paper [TODO: URL / Zenodo DOI].

Acknowledgements. [TODO: Acknowledgements: computing centres (Ifremer Datarmor, the hpc cluster, TGCC Irène under projects gen17863 and gen17889), funding.] This work used HEALPix (Górski et al. 2005), healpy (Zonca et al. 2019), NumPy (Harris et al. 2020), SciPy (Virtanen et al. 2020), Matplotlib (Hunter 2007), rsmpi (Steinbusch et al. 2026), cudarc (Lowman 2026) and cdshealpix (Pineau 2026); the reference implementation is STL-Dev (STL-Proj 2025), with the anamorphosis of FOSCAT (Delouis 2022). MPI is the standard of Message Passing Interface Forum (2023).

References

Allys, E., Levrier, F., Zhang, S., et al. 2019, A&A, 629, A115
Allys, E., Marchand, T., Cardoso, J.-F., et al. 2020, Phys. Rev. D, 102, 103506
Bi, K., Xie, L., Zhang, H., et al. 2023, Nature, 619, 533
Bradbury, J., Frostig, R., Hawkins, P., et al. 2018, JAX: composable transformations of Python+NumPy programs, `http://github.com/jax-ml/jax`, citation from the README of the repository

Bruna, J. & Mallat, S. 2013, IEEE Transactions on Pattern Analysis and Machine Intelligence, 35, 1872
Bruna, J. & Mallat, S. 2018, Mathematical Statistics and Learning, 1, 257
Campeti, P., Delouis, J.-M., Pagano, L., et al. 2025, A&A, 700, A136
Cheng, S. & Ménard, B. 2021, arXiv e-prints, arXiv:2112.01288
Cheng, S., Morel, R., Allys, E., Ménard, B., & Mallat, S. 2024, PNAS Nexus, 3, pgae103
Cheng, S., Ting, Y.-S., Ménard, B., & Bruna, J. 2020, MNRAS, 499, 5902
Delouis, J.-M. 2022, FOSCAT: generate synthetic Healpix or 2D data using Cross Scattering Transform, https://github.com/jmdelouis/FOSCAT, python package foscat on PyPI (https://pypi.org/project/foscat/), first released in 2022 (version 0.1); latest release 2026.6.1
Delouis, J.-M., Allys, E., Gauvrit, E., & Boulanger, F. 2022, A&A, 668, A122
Euclid Collaboration: Ajani, V., Baldi, M., Barthelemy, A., et al. 2023, A&A, 675, A120
Eyring, V., Bony, S., Meehl, G. A., et al. 2016, Geoscientific Model Development, 9, 1937
Frazer, W. D. & McKellar, A. C. 1970, Journal of the ACM, 17, 496
Fritsch, F. N. & Carlson, R. E. 1980, SIAM Journal on Numerical Analysis, 17, 238
Gatti, M., Jeffrey, N., Whiteway, L., et al. 2024, Phys. Rev. D, 109, 063534
Gatys, L. A., Ecker, A. S., & Bethge, M. 2015, in Advances in Neural Information Processing Systems, ed. C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, & R. Garnett, Vol. 28 (Curran Associates, Inc.)
Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, ApJ, 622, 759
Harris, C. R., Millman, K. J., van der Walt, S. J., et al. 2020, Nature, 585, 357
Hersbach, H., Bell, B., Berrisford, P., et al. 2020, Quarterly Journal of the Royal Meteorological Society, 146, 1999
Hewitt, H. T., Roberts, M., Mathiot, P., et al. 2020, Current Climate Change Reports, 6, 137
Hunter, J. D. 2007, Computing in Science & Engineering, 9, 90
Jeffrey, N., Boulanger, F., Wandelt, B. D., et al. 2022, MNRAS, 510, L1
Karlbauer, M., Cresswell-Clay, N., Durran, D. R., et al. 2024, Journal of Advances in Modeling Earth Systems, 16, e2023MS004021
Kochkov, D., Yuval, J., Langmore, I., et al. 2024, Nature, 632, 1060
Kratochvil, J. M., Haiman, Z., & May, M. 2010, Phys. Rev. D, 81, 043519
Kratochvil, J. M., Lim, E. A., Wang, S., et al. 2012, Phys. Rev. D, 85, 103513
Lam, R., Sanchez-Gonzalez, A., Willson, M., et al. 2023, Science, 382, 1416
Liu, D. C. & Nocedal, J. 1989, Mathematical Programming, 45, 503
Lowman, C. 2026, cudarc: safe and minimal CUDA bindings, https://github.com/chelsea0x3b/cudarc, rust crate cudarc, version 0.19.10 (https://crates.io/crates/cudarc); repository formerly at github.com/coreylowman/cudarc
Mallat, S. 2012, Communications on Pure and Applied Mathematics, 65, 1331
McEwen, J. D., Wallis, C. G. R., & Mavor-Parker, A. N. 2022, in International Conference on Learning Representations (ICLR 2022)
Message Passing Interface Forum. 2023, MPI: A Message-Passing Interface Standard, Version 4.1, https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf, approved by the MPI Forum on 2023 November 2
Morel, R., Rochette, G., Leonarduzzi, R., Bouchaud, J.-P., & Mallat, S. 2025, Applied and Computational Harmonic Analysis, 75, 101724
Mousset, L., Allys, E., Price, M. A., et al. 2024, A&A, 691, A269
Nocedal, J. & Wright, S. J. 2006, Numerical Optimization, 2nd edn., Springer Series in Operations Research and Financial Engineering (New York: Springer)
Paszke, A., Gross, S., Massa, F., et al. 2019, in Advances in Neural Information Processing Systems, ed. H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, & R. Garnett, Vol. 32 (Curran Associates, Inc.)
Petri, A., Haiman, Z., Hui, L., May, M., & Kratochvil, J. M. 2013, Phys. Rev. D, 88, 123002
Pineau, F.-X. 2026, cdshealpix: the CDS HEALPix library in Rust, https://github.com/cds-astro/cds-healpix-rust, rust crate cdshealpix, version 0.10.1 (https://crates.io/crates/cdshealpix); Centre de Données astronomiques de Strasbourg
Portilla, J. & Simoncelli, E. P. 2000, International Journal of Computer Vision, 40, 49
Price, I., Sanchez-Gonzalez, A., Alet, F., et al. 2025, Nature, 637, 84
Regaldo-Saint Blancard, B., Allys, E., Boulanger, F., Levrier, F., & Jeffrey, N. 2021, A&A, 649, L18
Schmalzing, J. & Górski, K. M. 1998, MNRAS, 297, 355
Smith, B. F., Bjørstad, P. E., & Gropp, W. D. 1996, Domain Decomposition: Parallel Multilevel Methods for Elliptic Partial Differential Equations (Cambridge: Cambridge University Press)
Steinbusch, B., Gaspar, A., & Brown, J. 2026, rsmpi: MPI bindings for Rust, https://github.com/rsmpi/rsmpi, rust crate mpi, version 0.8.2 (https://crates.io/crates/mpi)
Stevens, B., Satoh, M., Auger, L., et al. 2019, Progress in Earth and Planetary Science, 6, 61
STL-Proj. 2025, STL-Dev: Scattering Transform Library (development repository), https://github.com/STL-Proj/STL-Dev, public GitHub repository of the STL-Proj organisation, created in December 2025; documentation at https://stl-dev.readthedocs.io; accessed 2026 October 27
Takahashi, R., Hamana, T., Shirasaki, M., et al. 2017, ApJ, 850, 24
Tomita, H. 1986, Progress of Theoretical Physics, 76, 952
Villani, C. 2009, Grundlehren der mathematischen Wissenschaften, Vol. 338, Optimal Transport: Old and New (Berlin, Heidelberg: Springer)
Virtanen, P., Gommers, R., Oliphant, T. E., et al. 2020, Nature Methods, 17, 261
Wolfe, P. 1969, SIAM Review, 11, 226
Zonca, A., Singer, L. P., Lenz, D., et al. 2019, Journal of Open Source Software, 4, 1298

Appendix A: Minkowski functionals on HEALPix

For a map u with mean removed and standard deviation σ_0, $x = u/\sigma_0$, the Minkowski functionals of the excursion sets $\{x > \nu\}$ on the unit sphere are (Schmalzing & Górski 1998)

$$V_0(\nu) = \frac{1}{4\pi}\int \Theta(x-\nu)\,\mathrm{d}\Omega, \tag{A.1}$$

$$V_1(\nu) = \frac{1}{4\pi}\int \frac{1}{4}\,\delta(x-\nu)\,|\nabla x|\,\mathrm{d}\Omega, \tag{A.2}$$

$$V_2(\nu) = \frac{1}{4\pi}\int \frac{1}{2\pi}\,\delta(x-\nu)\,\frac{2x_{;\theta}x_{;\phi}x_{;\theta\phi} - x_{;\theta}^2 x_{;\phi\phi} - x_{;\phi}^2 x_{;\theta\theta}}{x_{;\theta}^2 + x_{;\phi}^2}\,\mathrm{d}\Omega, \tag{A.3}$$

with the covariant derivatives $x_{;\theta} = \partial_\theta x$, $x_{;\phi} = \partial_\phi x/\sin\theta$, $x_{;\theta\theta} = \partial_\theta x_{;\theta}$, $x_{;\theta\phi} = \partial_\theta x_{;\phi}$ and $x_{;\phi\phi} = \partial_\phi x_{;\phi}/\sin\theta + \cot\theta\, x_{;\theta}$. V_2 is the integral of the geodesic curvature of the contours; by the Gauss–Bonnet theorem it differs from the Euler characteristic of the excursion set by an area term, and its Gaussian expectation has no curvature correction on the sphere. For a Gaussian field with $\tau = \langle|\nabla x|^2\rangle/2$ (Tomita 1986),

$$V_0 = \tfrac{1}{2}\mathrm{erfc}\left(\nu/\sqrt{2}\right), \quad V_1 = \tfrac{1}{8}\sqrt{\tau}\,e^{-\nu^2/2}, \quad V_2 = \frac{\tau\,\nu\,e^{-\nu^2/2}}{(2\pi)^{3/2}}, \tag{A.4}$$

and $\tau = \sum_\ell (2\ell+1)\ell(\ell+1)C_\ell/\left(2\sum_\ell(2\ell+1)C_\ell\right)$.

`tools/minkowski.py` computes the derivatives in harmonic space: `healpy.alm2map_der1` gives $x_{;\theta}$ and $x_{;\phi}$ from the harmonic coefficients of the (optionally smoothed) map, and applied to the harmonic coefficients of these two maps it gives the second derivatives. The Dirac distribution is estimated by the fraction of pixels in bins of width $\Delta\nu = 0.2$. The test of the tool (`tools/test_minkowski.py`) draws Gaussian random fields at $N_{\mathrm{side}} = 256$ with a white spectrum smoothed over 4 pixels and compares them with the expressions above, for the τ measured on each field. The largest deviation per field is 0.2, 1.1 and 3.1% of the maximum of V_0, V_1 and V_2; for the mean of four fields 0.1, 0.4 and 1.3%; the measured τ agrees with the spectrum's to 0.06%. A lognormal field of the same spectrum is flagged (deviations of 71% and 100% of the maxima of V_1 and V_2). For the convergence maps we use thresholds $-3 \le \nu \le 6$ after a Gaussian smoothing over 4 pixels of the map's resolution and over 30 arcmin.